

Adaptive genetic algorithm for parcel hub scheduling problem with shortcuts in closed-loop sortation system

James C. Chen^a, Tzu-Li Chen^{b,*}, Ting-Chieh Ou^a, Yu-Hsin Lee^a

^a Department of Industrial Engineering and Engineering Management, National Tsing Hua University, Hsinchu 30013, Taiwan, ROC

^b Department of Information Management, Fu Jen Catholic University, New Taipei City 24205, Taiwan, ROC

ARTICLE INFO

Keywords:

Parcel hub scheduling problem
Shortcuts
Adaptive genetic algorithm
Local search
Fuzzy logic control
Closed-loop sortation system

ABSTRACT

This paper focuses on the parcel hub scheduling problem with shortcuts (PHSPwS), which is a critical inbound scheduling problem in parcel delivery industries such as postal service. PHSPwS employs shortcuts to determine the unloading schedule of inbound trailers/trucks in the closed-loop sorting process to minimize total makespan. PHSPwS not only considers the unequal batch sizes and various arrival times of inbound trailers but most importantly, the alternative parcel routing caused by the existence of multiple shortcuts in the closed-loop sortation system. A non-linear mixed integer program model is first formulated to address this problem. Given the computational complexity of the developed model, this research further proposes an adaptive genetic algorithm (AGA) to solve PHSPwS effectively. In the proposed AGA, local search (LS) is adopted to avoid entrapment in the local optimal solution, whereas fuzzy logic control (FLC) is utilized to adjust the probability of crossover and mutation rates. Considering the changes in average fitness values of parents and offspring in two consecutive generations could enhance the searching capability of the proposed algorithm. Computational results show that AGA with LS and FLC performs the solution better and with more stability than the other algorithms.

1. Introduction

Logistics is the key to modern economy. From the steel factories of Pennsylvania to the port of Singapore and from the Nicaraguan banana fields to postal delivery and solid waste collection in any region of the world, almost every organization is challenged with the timely transport of materials to accurate locations (Ghani, Laporte, & Musmanno, 2004). Logistics plays an important role in supply chain management, and considerable attention has been paid to such issues recently. Given the importance of logistics, numerous companies are attempting to develop efficient ways to increase customer satisfaction and reduce spending costs. Managers have realized that delivering their products to customers faster than competitors will substantially enhance their market competitiveness. Hence, companies must search for effective approaches to strengthen their relative capabilities, such as distribution network, load planning, and scheduling.

In parcel delivery network, the hubs of a hub-and-spoke configuration are the central parcel consolidation terminals (CPCTs). CPCTs link the numerous origin-to-destination nonhub pairs by requiring few transportation links (McWilliams, 2010a). In the current work, a closed-loop sortation system built from automated conveyors is involved

between the receipt (inbound) and shipment (outbound) activities within the CPCTs. Parcels in inbound trailers must be unloaded at unloading docks, sorted across a fixed network of conveyors, and transferred to outbound trailers at loading docks. Fig. 1 shows a typical CPCT. The deep-gray rectangles represent inbound trailers, whereas the white rectangles represent outbound trailers. Fig. 1 also displays a queue of inbound trailers waiting to be serviced at the unloading docks. In certain realistic companies, the material handling system, as shown in the middle of Fig. 1, may also build several conveyor segments or “shortcuts” to connect unloading and loading docks. Thus, the pace and efficiency of parcel sorting flows are improved, and the throughput of whole sortation system is increased.

Scheduling of inbound and outbound trailers is one of the most popular and critical research issues in fully automated sorting systems of distribution centers (Boysen, Briskorn, Fedtke, & Schmickerath, 2018). However, this research only considers the schedule of inbound trailers as it assumes that outbound trailers are already scheduled. As the assignment and sequence of inbound trailers to unloading docks affect the amount of congestion in the transfer system (i.e., the fixed network of conveyors) and timespan of transfer operation, the schedule of inbound trailers becomes a critical problem. This inbound scheduling

* Corresponding author.

E-mail address: chentzuli@gmail.com (T.-L. Chen).

<https://doi.org/10.1016/j.cie.2019.106114>

Received 13 April 2019; Received in revised form 19 September 2019; Accepted 5 October 2019

Available online 18 October 2019

0360-8352/ © 2019 Elsevier Ltd. All rights reserved.

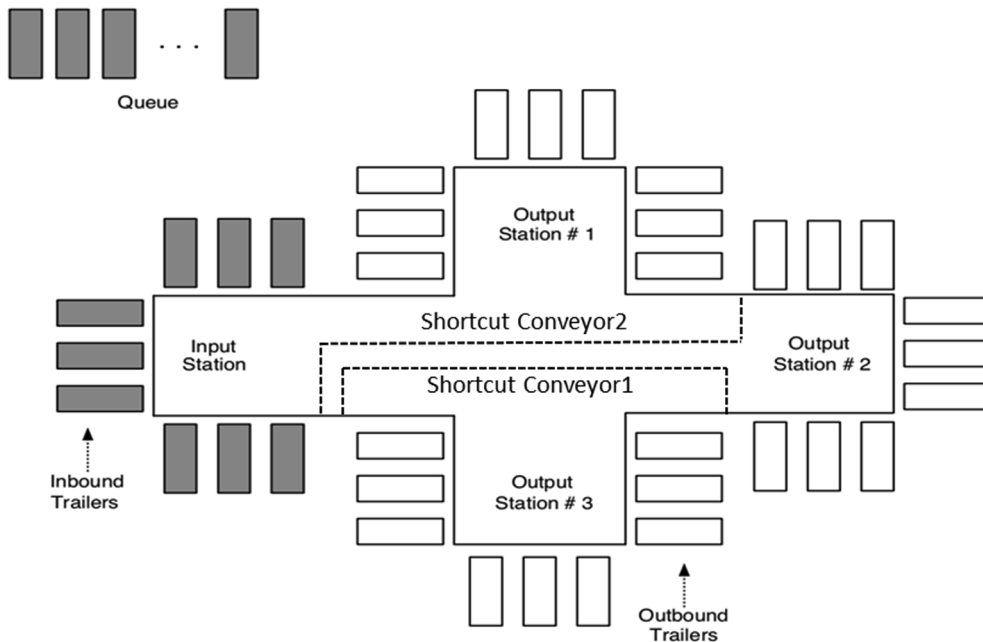


Fig. 1. A central parcel consolidation terminal with the shortcut conveyors.

problem is known as the parcel hub scheduling problem (PHSP) problem, which was first formally introduced by [McWilliams, Stanfield, and Geiger \(2005\)](#). Most previous works studied the PHSP that only considers closed-loop sortation systems without shortcuts ([McWilliams & McBride, 2012](#); [McWilliams et al., 2005](#); [McWilliams, 2009a, 2009b, 2010a, 2010b](#); [McWilliams, Stanfield, & Geiger, 2008](#)). However, as demonstrated by the realistic cases in [Fig. 1](#), construction of a “shortcut” will improve the speed and efficiency of parcel sorting flows and increase throughput of the whole system. To the best of our knowledge, no literature studied the inbound scheduling problem of the “closed-loop sorting process with shortcuts.” Therefore, the current work is the first to investigate the PHSP with shortcuts (PHSPwS) on a closed-loop sortation system. Given the existence of multiple shortcuts in a closed-loop sortation system, alternative parcel routings from unloading to loading docks must be considered to increase the difficulty of solving PHSPwS in comparison with PHSP. In addition, travel time and traffic congestion of different parcel routes heavily affect the completion time of the sorting process.

This research focuses on the PHSPwS, which is a critical inbound scheduling problem typically observed in parcel delivery industries such as postal service ([Boysen et al., 2018](#)). PHSPwS is a combinatorial optimization problem that involves assigning a high number of inbound trailers to a considerably lower number of unloading docks to minimize the time needed to process inbound trailers. Each inbound trailer contains a batch of heterogeneous parcels. Heterogeneous parcels include parcels that are bound for different destinations in the delivery network. Thus, they must be loaded, sorted, and unloaded onto appropriate outbound trailers heading to various destinations. Timespan is defined as the combination of moving and stoppage times. Moving time is the actual time required to transport parcels from unloading to loading docks via the conveyor system, whereas stoppage time is the congestion-induced delay ([McWilliams, 2010a](#)).

This study aims to find the optimal unloading schedule for inbound trailers/trucks in the closed-loop sorting process with shortcuts to maximum completion time. Assignment and sequence of inbound trailers to unloading docks may impact the amount of congestion in the sortation system and thus the makespan of sorting operation. A considerable unloading schedule can accelerate the sorting process and enhance CPCT performance. To recognize the optimal unloading schedule for inbound trailers, a nonlinear mixed-integer problem is

formulated to address the PHSPwS problem. Furthermore, as a result of the computational complexity and nonlinear nature of the developed model, this research proposes an adaptive genetic algorithm (AGA). The AGA combines fuzzy logic control (FLC) and local search (LS) with the genetic algorithm (GA) to generate an unloading schedule for the PHSPwS effectively. In the proposed AGA, LS is adopted to avoid entrapment in the local optimal solution. FLC is utilized to adjust the probability of crossover and mutation rates. Considering the changes in average fitness values of parents and offspring in two consecutive generations could enhance the searching capability of the proposed algorithm.

The remainder of this paper is organized as follows. [Section 2](#) reviews the related literature. [Section 3](#) describes the PHSPwS problem. [Section 4](#) formulates PHSPwS as a nonlinear mixed integer program model. [Section 5](#) develops the AGA combining FLC and LS to solve the PHSPwS problem effectively. [Section 6](#) demonstrates the efficiency of the developed AGA through numerical experiments. Finally, [Section 7](#) provides the conclusions drawn from this research and the proposed directions for future studies.

2. Literature review

Improving the operational performance of CPCT failed to attract substantial attention from previous research. [Rohrer \(1995\)](#) discussed the significance of CPCT modeling. [Masel and Goldsmith \(1997\)](#) and [Masel \(1998\)](#) proposed other procedures to designate loading dock destinations. Historical data from previous studies were used to balance parcel flow to loading docks at the static level. [Lai \(2006\)](#) proposed a surrogate search (SS) method that combines minimum simulation evaluations with a regression model to predict system behavior. Instead of minimizing the timespan of transfer operation by focusing on controlling the input of parcel types at unloading docks, this method reacts to parcel workflow variation by moving teams of loaders to terminal areas impacted by high parcel workflow in the facility. The results demonstrate that SS can be applied to such large-scale simulation problems and outperform recognized simulation optimization methodology.

The studies of [McWilliams et al. \(2005\)](#) and [McWilliams et al. \(2008\)](#) utilized a simulation-based scheduling approach with GA to solve PHSP. [McWilliams et al. \(2005\)](#) considered a scheduling problem

commonly encountered by freight consolidation terminals in parcel delivery industry. The authors defined this problem as the PHSP and discussed the characteristics and challenges of solving this issue. To solve the PHSP, they proposed a simulation-based scheduling algorithm (SBSA) that employs GA to generate solutions. The effectiveness of SBSA was evaluated by applying the algorithm to three separate hub scheduling problems with different terminal sizes. McWilliams et al. (2008) discussed a similar problem but with unequal batch sizes in inbound trailers and proposed a simulation-based scheduling approach with an embedded GA.

In the works of McWilliams (2009b) and McWilliams (2010a), PHSP was modeled as a minimax problem, and a GA was developed to search for optimal solutions to the PHSP. McWilliams (2009b) proposed a mathematical model to minimize the maximum workload on parallel output stations to solve small-size problems. On the other hand, a GA was proposed to solve large-size problems with minimum computational time compared with SBSA and random scheduling. McWilliams (2010a) compared the performance of the proposed algorithm with that of an existing algorithm, random scheduling, and a lower bound. The study showed that scheduling can significantly impact the operating performance of a parcel consolidation terminal, and that the cost tradeoff to generate unloading schedules can be minimized compared with potential savings in labor cost.

McWilliams (2010b) developed and evaluated LS and simulated annealing (SA) to determine optimal unloading schedules for PHSP. The study showed that iterative improvement with 2-opt is a superior search approach for solutions to the PHSP compared with other perturbation strategies and GA. The SA and LS algorithms with 2-opt provided better solutions than GA. McWilliams and McBride (2012) proposed a beam search heuristic to solve PHSP. The objective is to minimize the time-span of transfer operation, which consists of unloading all inbound trailers, sorting and moving all parcels to appropriate unloading docks, and loading all parcels to outbound trailers. The performance of the heuristic is compared with that of GA, simulation-based scheduling, and random scheduling. McWilliams (2009a) proposed a dynamic load-balancing algorithm (DLBA) to solve PHSP. DLBA constructs unloading schedules for transfer operation using a recursive procedure. The performance of the algorithm was compared with that of an efficient static load-balancing algorithm and an existing lower bound.

From the above reviews, previous literature almost studied parcel hub scheduling in closed-loop sortation systems without considering shortcut and alternative parcel routings. To the best of our knowledge, we are the first to study the PHSPwS problem on a closed-loop sortation system. Therefore, the current study focuses on finding an unloading schedule for inbound trailers with various batch sizes and arrival time by minimizing the completion time of closed-loop sorting process with shortcuts and alternative parcel routings. Given the computational complexity and nonlinear nature of PHSPwS, an AGA utilizing FLC and LS mutator is also developed to deal with large-scale instances effectively.

3. Problem definition and system description

This research mainly focuses on solving problems with the following situation: when a set of inbound trailers containing numerous parcels arrive at the cross-dock facility, inbound trailers are scheduled to be unloaded at unloading docks. The conveyor system in the facility is equipped with shortcuts to rapidly sort parcels. This kind of problem is referred to as PHSPwS. The closed-loop conveyor system shown in Fig. 2 comprises a mainline conveyor, shortcuts that serve as highways of the mainline conveyor, induction line conveyors that connect unloading docks with the mainline conveyor, and chutes that connect loading docks with the mainline conveyor. The induction line conveyor, chutes, and shortcuts cannot connect to one another. A conveyor network with various routes could be established by configuring these elements in the system.

In this research, an algorithm with time-based decomposition is used to solve the PHSPwS with unequal batch sizes and various arrival times of inbound trailers. Hence, the time horizon is divided into time slots. The following assumptions are implemented in this research:

- All unloading docks in the conveyor system are identical. Unloading docks can process any inbound trailer, and they feature equal and constant processing rates.
- All loading docks in the conveyor system are identical. Loading docks can process any parcel, and they feature equal and constant processing rates.
- Inbound trailers exhibit various arrival times and batch sizes.
- Empty inbound trailers are instantaneously replaced with full inbound trailers.
- Full outbound trailers are instantaneously replaced with empty outbound trailers.
- All inbound and outbound trailers present equal priority when available.
- Every unloading dock can unload one inbound trailer segment per time slot.

Table 1 shows the notations, including indices, sets, parameters, and decision variables, that will be used in the mathematical model (Section 4).

Fig. 2 illustrates a typical conveyor system. The conveyor system contains one mainline conveyor (M1 to M14), four shortcuts (C1, C2, C3, and C4), two induction line conveyors, and four chutes. Each induction line conveyor aids in transporting parcels that are unloaded from inbound trailers at unloading docks (U1 and U2). Each chute helps in transporting sorted parcels from the mainline conveyor to loading docks (L1, L2, L3, and L4). The conveyor system is divided into several segments by unloading docks, loading docks, and shortcuts. The quantity of conveyor segments in a conveyor system is the sum of the number of unloading docks plus the number of loading docks and plus thrice the number of shortcuts. In this case, the quantity of conveyor segments equals 18. Moreover, each conveyor segment features various lengths (TS_m) that are counted in time slots.

The combination of conveyor segments can create a number of routes. Every counter-clockwise path between unloading and load docks creates a route. A route cannot contain duplicate conveyor segments (i.e., parcels cannot pass through one conveyor segment twice). Thus, all routes in the system can construct a conveyor system network. Fig. 3 shows the network of the conveyor system. Fourteen routes could be created in this conveyor system. For instance, one route exists between U1 and L1, and it contains conveyor segments M1, M2, M3, and M4 (i.e., every parcel that is unloaded at U1 and is assigned to L1; it can only pass through the route that includes conveyor segments M1, M2, M3, and M4). However, when a parcel is unloaded at U1 and is assigned to L4, three routes could be selected as transport paths (i.e., R5, R6, and R7). Among these routes, R7 is the longest. R7 contains conveyor segments M1, M2, M3, M4, M5, M6, M7, M8, M9, M10, M11, M12, and M13. Notably, R7 transports parcels by the mainline conveyor. R6 contains conveyor segments M1, M2, C2, M11, M12, and M13. This route utilizes one of the shortcuts in the system. As a result, parcels require less time slots to travel along R6. R5 contains conveyor segments M1, C1, and M13. Hence, parcels require the shortest time slots to travel along R5. In consequence, we prefer to transfer parcels via R5 rather than R6 and R7 owing to the reduced transfer time.

Parcels from around the world are sent to cross-dock facilities by inbound trailers. As every time slot can be used to process pre-determined quantity of parcels and unequal batch sizes among inbound trailers, every inbound trailer is partitioned into segments. Each segment could be processed in a single time slot. For inbound trailers that need to be processed for more than one time slot, the parcel types and quantities are uniformly distributed over the segments. Eq. (1) defines the number of time slots required to process an inbound trailer. For

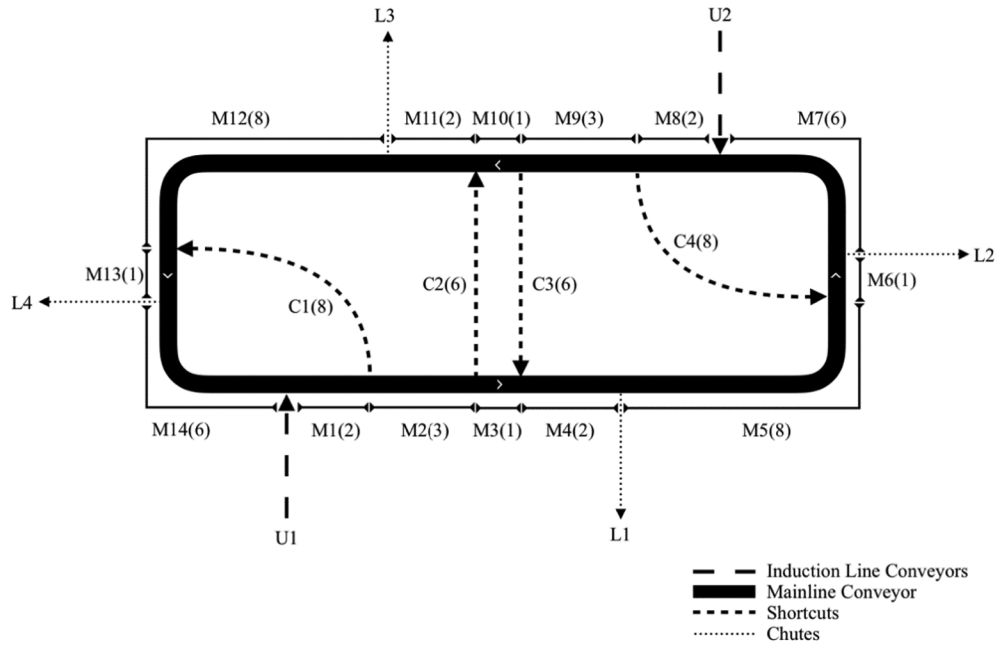


Fig. 2. Illustration of the conveyor system.

Table 1

Notations of the mathematical mode.

● Indices	
u	Index value for the unload docks.
l	Index value for the load docks.
i	Index value for the inbound trailers.
k	Index value for the inbound trailer segments.
r	Index value for the routes from unload dock to load dock.
j	Index value for the time slots.
m	Index value for the conveyor segments (including mainline conveyor segments and shortcuts).
● Sets	
U	Set of unload docks.
L	Set of load docks.
J	Set of time slots.
I	Set of inbound trailers.
M	Set of conveyor segments.
M_{ulr}	Ordered set of conveyor segments of route r from unload dock u to load dock l ($r \in R_{ul}$, $M_{ulr} \subseteq M$), where $M_{ulrf(i)}$ is the first and $M_{ulrre(i)}$ is the end element of M_{ulr} .
R_{ul}	Set of routes between unload dock u to load dock l .
K_i	Set of segments in inbound trailer i .
● Parameters	
A_i	Arrival time slot of the inbound trailer i .
N_{ikl}	Quantity of parcels in inbound trailer i segment k that need to be loaded to the outbound trailer at load dock l .
S_i	Number of time slots required to process inbound trailer i .
Q	Quantity of parcels that can be unloaded from an inbound trailer at each unload dock in a time slot.
MF_m	Maximum flow limitation of conveyor segment m in a time slot.
TS_m	Length of conveyor segment m , i.e., requirement of time slots for a parcel to pass through conveyor segment m .
RL_{ulr}	Length of route r which starts from unload dock u and end at load dock l , i.e., requirement of time slots for a parcel to pass through route r .
Z	A large number.
● Decision variables	
X_{iku}	Binary variable: 1, if inbound trailer i segment k is assigned to unload dock u in time slot j ; 0, otherwise.
RS_{ikulr}	Binary variable: 1, if inbound trailer i segment k selects route r which starts from unload dock u and ends at load dock l ; 0, otherwise.
QR_{ikulr}	Quantity of parcel from inbound trailer i segment k that will pass through route r which starts from unload dock u and ends at load dock l .
AT_{ikmulr}	First time slot of parcels in inbound trailer i segment k to arrive at conveyor segment m that is part of route r which starts from unload dock u and ends at load dock.
PA_{mjulr}	Quantity of the parcels first arrive at conveyor segment m that is part of route r which starts from unload dock u and ends at load dock in time slot j .
$PK_{ikmjulr}$	Quantity of the parcels from inbound trailer i segment k remain at conveyor segment m that is part of route r which starts from unload dock u and ends at load dock l after arriving conveyor segment m in a time slot precedes time slot j .
PB_{mjulr}	Quantity of the parcels remain at conveyor segment m that is part of route r which starts from unload dock u and ends at load dock l after arriving conveyor segment m in a time slot precedes time slot j .
PF_{mjulr}	Parcel flow at conveyor segment m in time slot j caused by route r which starts from unload dock u and ends at load dock l .
TF_{mj}	Total parcel flow at conveyor segment m in time slot j .
CT_{ikulr}	Completion time slot of parcels in inbound trailer i segment k that are transported via route r which starts from unload dock u and ends at load dock l , i.e., the time slot when parcels with the same route arrive to its predetermined load dock.
C_{max}	Maximum completion time slot over all load docks (makespan).

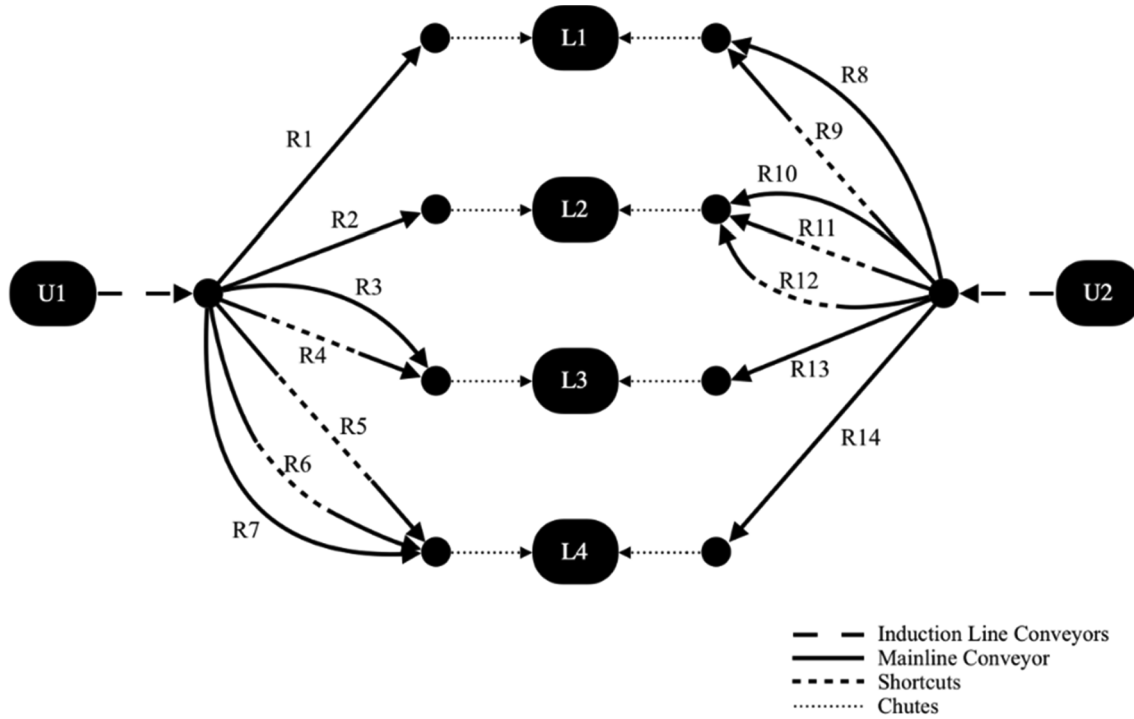


Fig. 3. Network representation of the conveyor system.

illustration, the quantity of parcels in every inbound trailer is 10, 20, or 30. The number of parcels (Q) that can be unloaded from an inbound trailer at each unloading dock per time slot is 10. As a result, the number of time slots (S_i) required to process each inbound trailer at unloading docks is 1, 2, or 3 (i.e., the number of segments for each inbound trailer is 1, 2, or 3, respectively).

$$S_i = \frac{\sum_{k \in K_i} \sum_{l \in L} N_{ikl}}{Q} \quad \forall i \in I \quad (1)$$

Fig. 4 shows an example of an inbound trailer containing 30 parcels.

The inbound trailer delivering 9 parcels that should be transported to loading dock 1, 12 parcels that should be transported to loading dock 2, and 9 parcels that should be transported to loading dock 3. In this research, the inbound trailer is partitioned into three segments. Every segment contains three parcels that should be transported to loading dock 1, four parcels that should be transported to loading dock 2, and three parcels that should be transported to loading dock 3. By calculating Eq. (1), the number of time slots required to process the inbound trailer is 3.

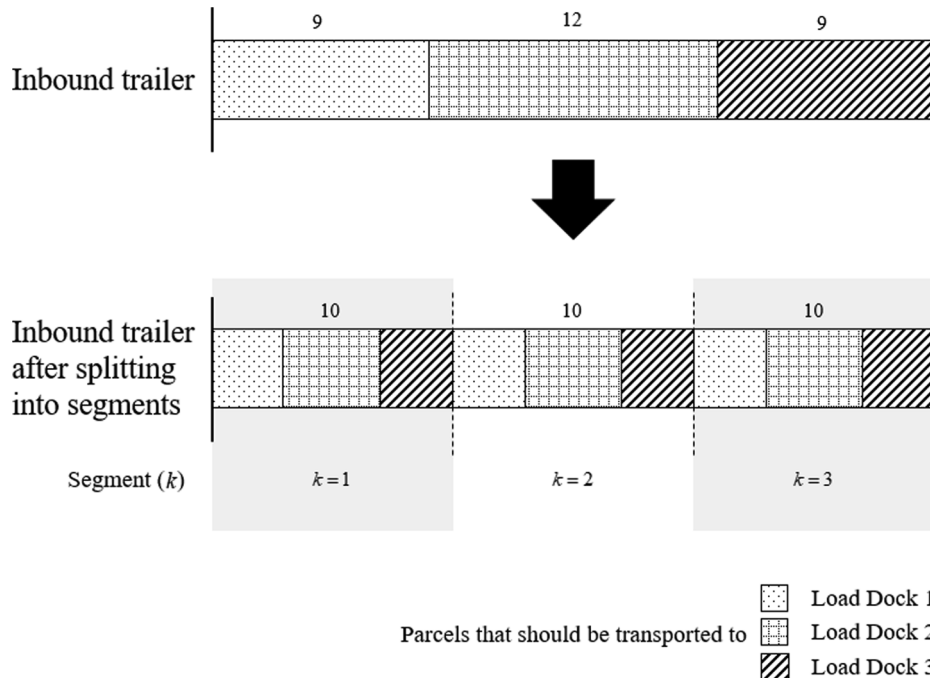


Fig. 4. Illustration of the inbound trailer.

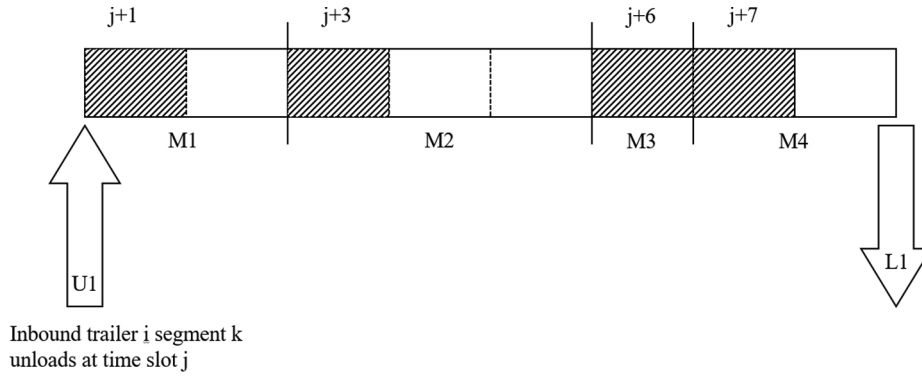


Fig. 5. Illustration of partial enlargement of the route between U1 and L1.

4. Mathematical model

This research aims to minimize the maximum completion time of the sorting process. The following mathematical model is utilized to find the optimal unloading schedule for inbound trailers and alternative route for parcels:

Minimize $C_{\max}$

Subject to

$$\sum_{i \in I} \sum_{k \in K_i} X_{ikuj} \leq |U| \quad \forall u \in U, j \in J \quad (2)$$

$$\sum_{u \in U} \sum_{j \in J} X_{ikuj} = 1 \quad \forall i \in I, k \in K_i \quad (3)$$

$$\sum_{u \in U} \sum_{j \in J} X_{ikuj} \times j \geq A_i \quad \forall i \in I, k \in K_i \quad (4)$$

$$S_i \times X_{i1uj} - \sum_{k \in K_i} X_{iku(j+k-1)} = 0 \quad \forall i \in I, u \in U, j \in J \quad (5)$$

$$\sum_{r \in R_{ul}} RS_{ikulr} = 1 \quad \forall i \in I, k \in K_i, u \in U, l \in L \quad (6)$$

$$RS_{ikulr} \leq \sum_{j \in J} X_{ikuj} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (7)$$

$$RS_{ikulr} \times Z \geq QR_{ikulr} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (8)$$

$$\sum_{u \in U} \sum_{r \in R_{ul}} RS_{ikulr} \times QR_{ikulr} = N_{ikl} \quad \forall i \in I, k \in K_i, l \in L \quad (9)$$

$$AT_{ikmulr} = \left(\sum_{\substack{m' \in M_{ulr} \\ m' < m}} TS_{m'} + \sum_{u \in U} \sum_{j \in J} X_{ikuj} \times j + 1 \right) \times RS_{ikulr} \quad \forall i \in I, k \in K_i, m \in M_{ulr}, u \in U, l \in L, r \in R_{ul} \quad (10)$$

$$PA_{mjulr} = RS_{ikulr} \times QR_{ikulr} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr}, j = AT_{ikmulr} \quad (11)$$

$$PK_{ikm(j+b)ulr} = RS_{ikulr} \times QR_{ikulr} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr}, j = AT_{ikmulr}, b \in \mathbb{Z}; 0 \leq b < TS_m \quad (12)$$

$$PB_{mjulr} = \sum_{i \in I} \sum_{k \in K_i} PK_{ikm(j+b)ulr} \quad \forall j \in J, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (13)$$

$$PF_{mjulr} = PA_{mjulr} + PB_{mjulr} \quad \forall j \in J, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (14)$$

$$TF_{mj} = \sum_{u \in U} \sum_{l \in L} \sum_{r \in R_{ul}} PF_{mjulr} \quad \forall j \in J; m \in M \quad (15)$$

$$TF_{mj} \leq MF_m \quad \forall j \in J, m \in M \quad (16)$$

$$CT_{ikulr} = \left(\sum_{u \in U} \sum_{j \in J} X_{ikuj} \times j + RL_{ulr} + 1 \right) \times RS_{ikulr} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (17)$$

$$C_{\max} = \max\{CT_{ikulr}\} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (18)$$

$$X_{ikuj} \in \{0, 1\} \quad \forall i \in I, k \in K_i, u \in U, j \in J \quad (19)$$

$$RS_{ikulr} \in \{0, 1\} \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (20)$$

$$QR_{ikulr} \geq 0 \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (21)$$

$$AT_{ikmulr} \geq 0 \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (22)$$

$$PA_{mjulr} \geq 0 \quad \forall j \in J, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (23)$$

$$PK_{ikm(j+b)ulr} \geq 0 \quad \forall i \in I, k \in K_i, j \in J, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (24)$$

$$PB_{mjulr} \geq 0 \quad \forall j \in J, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (25)$$

$$PF_{mjulr} \geq 0 \quad \forall j \in J, u \in U, l \in L, r \in R_{ul}, m \in M_{ulr} \quad (26)$$

$$TF_{mj} \geq 0 \quad \forall j \in J, m \in M \quad (27)$$

$$CT_{ikulr} \geq 0 \quad \forall i \in I, k \in K_i, u \in U, l \in L, r \in R_{ul} \quad (28)$$

$$C_{\max} \geq 0 \quad (29)$$

$$C_{\max} \geq 0 \quad (30)$$

In the objective function (2), $C_{\max}$ denotes the makespan of overall sorting processes. Eq. (3) ensures that no more than $|U|$ inbound trailer segments are assigned to a single time slot j as only $|U|$ unloading docks exist. Eq. (4) guarantees that inbound trailer i segment k is assigned to only one unloading dock in one time slot. Eq. (5) ensures that inbound trailer i segment k can only be processed upon arrival. The calculation in the left-hand side of this equation represents the time slot when inbound trailer i segment k is assigned to an unloading dock. Eq. (6) assures that the segments of a single inbound trailer are successively processed based on the assignment of the first inbound trailer segment. Therefore, the unloading process of an inbound trailer cannot be interrupted, and all segments in the same inbound trailer are unloaded in the same unloading dock. Eq. (7) ensures that parcels with the same destination (load dock l) in inbound trailer i segment k can only be transported via the same route. Eq. (8) sets RS_{ikulr} equal to zero if inbound trailer i segment k remains unassigned to any unloading dock u in any time slot j . Eq. (9) guarantees that if inbound trailer i segment k fails to utilize route r , which starts from unloading dock u and ends at loading dock l , no parcel will be transported via route r . Eq. (10) ensures that the total quantity of parcels in inbound trailer i segment k

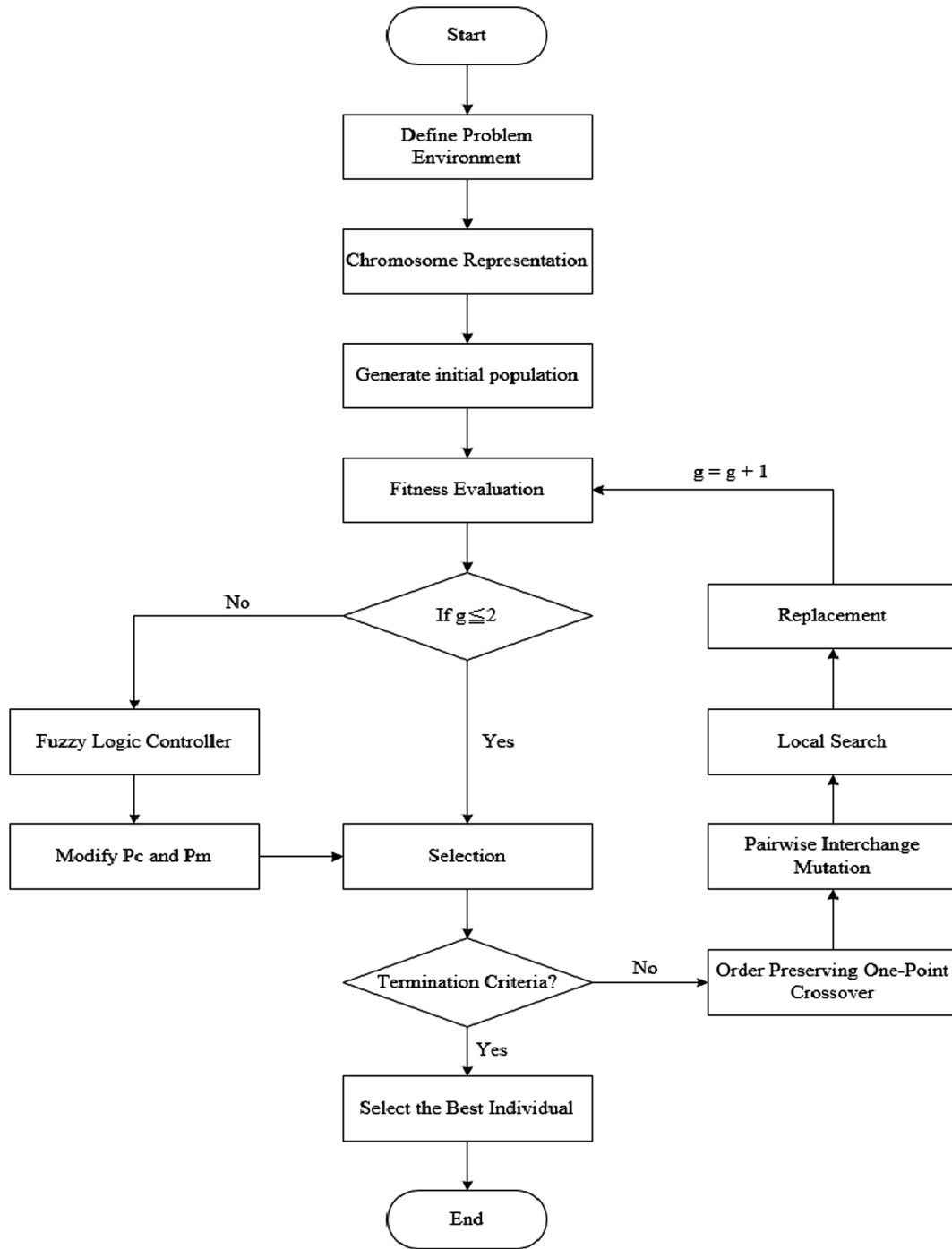


Fig. 6. Framework of the AGA with FLC and LS method.

Table 2
An example of initial population.

Chromosomes	Inbound Trailers Sequence
Chromosome 1	6-2-9-3-1-4-8-5-10-7
Chromosome 2	4-7-1-3-5-9-8-6-2-10
Chromosome 3	10-6-7-4-2-9-3-8-1-5
...	...
Chromosome 80	7-10-3-5-8-1-6-9-2-4

and which are assigned to loading dock l via route r equals the original quantity in inbound trailer i segment k , which needs to be assigned to loading dock l .

Eq. (11) calculates the decision variable AT_{ikmulr} . AT_{ikmulr} denotes the time slot when parcels from inbound trailer i segment k first arrive at conveyor segment m in route r which starts from unloading dock u and ends at loading dock l . Fig. 5 shows the illustration of partial enlargement of the route between U1 and L1. Every conveyor segment is partitioned into several time slots relative to their length (TS_m). Each rectangle represents one time slot. The rectangles covered with oblique lines are the decision variable AT_{ikmulr} . The only route R1 ($r = 1$) between U1 ($u = 1$) and L1 ($l = 1$) contains four conveyor segments: M1 ($m = 1$), M2 ($m = 2$), M3 ($m = 3$), and M4 ($m = 4$). We assume that inbound trailer i segment k is unloaded at time slot j , and several parcels will be transported via route R1. Hence, $j + 1$ is the time slot when parcels from inbound trailer i segment k first arrive at conveyor

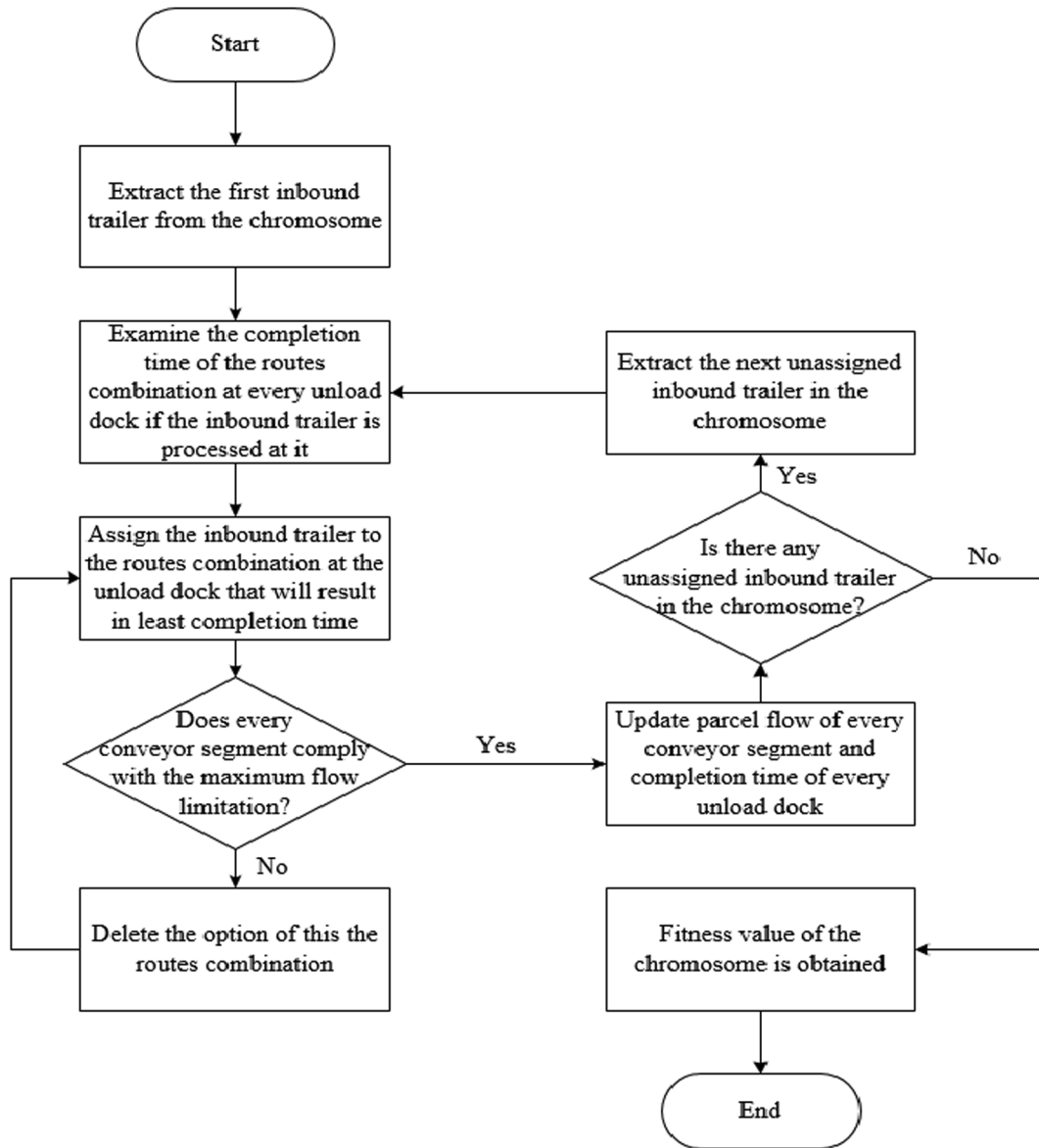


Fig. 7. Process of the fitness value evaluation.

segment M1 in route R1 (AT_{ik1111}). Furthermore, $j + 3$ is the time slot when parcels from inbound trailer i segment k first arrive at conveyor segment M2 in route R1 (AT_{ik2111}). AT_{ik3111} and AT_{ik4111} can be calculated as $j + 6$ and $j + 7$, respectively.

Eq. (12) to Eq. (16) calculate the decision variables PA_{mjulr} , $PK_{ikmjulr}$, PB_{mjulr} , PF_{mjulr} , and TF_{mj} . Eq. (17) provides a maximum flow limitation for conveyor segment m in a single time slot because a conveyor segment cannot handle extremely high numbers of parcels at a single time slot. In Eq. (18), completion time slot of parcels in inbound trailer i segment k that are transported via route r , which starts from unloading dock u and ends at loading dock l , is calculated (CT_{ikulr}). As we need to minimize the completion time of the sorting system, we need to find the final time slot of loading docks with parcels to be processed. The completion time of the whole conveyor system could be calculated by finding the latest CT_{ikulr} in Eq. (19). The following constraints are domain restrictions on decision variables. Eqs. (20) and (21) ensure that X_{ikuj} and RS_{ikulr} use binary values, respectively. Eqs. (22)–(30) assure that the decision variables are greater or equal to zero.

5. Adaptive genetic algorithm

5.1. GA with FLC and LS

In this paper, a LS strategy proposed by Ombuki and Ventresca (2004) is probabilistically applied by replacing the mutation operator in AGA with an improved mutation operator aimed at LS. Fig. 6 shows the framework of the AGA with FLC and LS method.

5.2. Initial population, fitness value evaluation, and selection

This paper sets 80 as the level of population size. Table 2 shows an illustration of the initial population. In this research, the makespan of each chromosome is observed and used to measure chromosome fitness. All chromosomes in the initial population are evaluated for their fitness value. Fig. 7 shows the flow chart of fitness value evaluation for one chromosome. We can obtain the chromosome fitness value by extracting the latest completion time of inbound trailers. However, every TPF_{mj} should not be larger than the maximum flow limitation.

The selection of chromosomes will be processed after chromosome evaluation. First, the chromosomes with the top 10% highest fitness

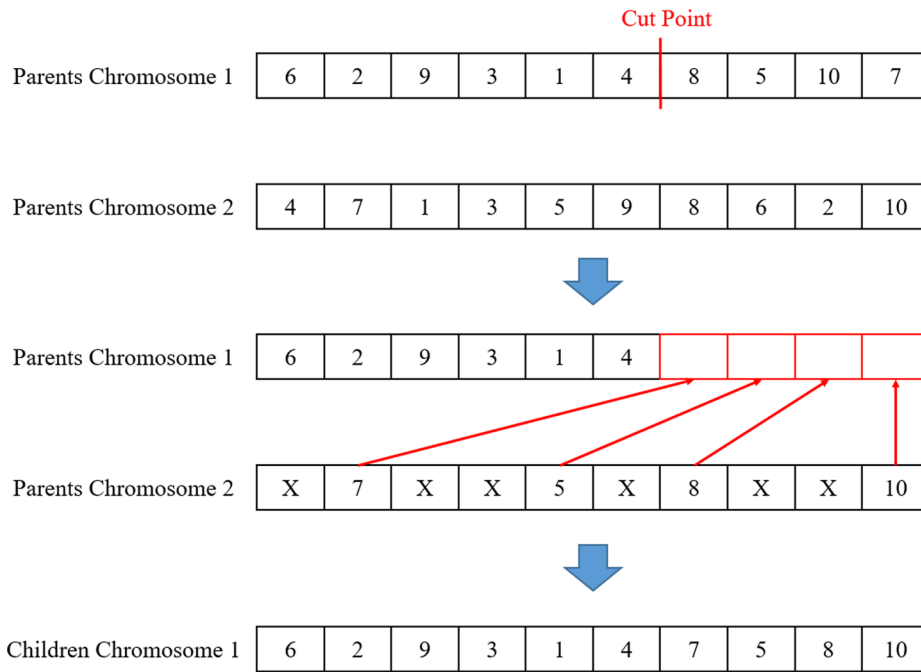


Fig. 8. An example of order preserving one-point crossover.

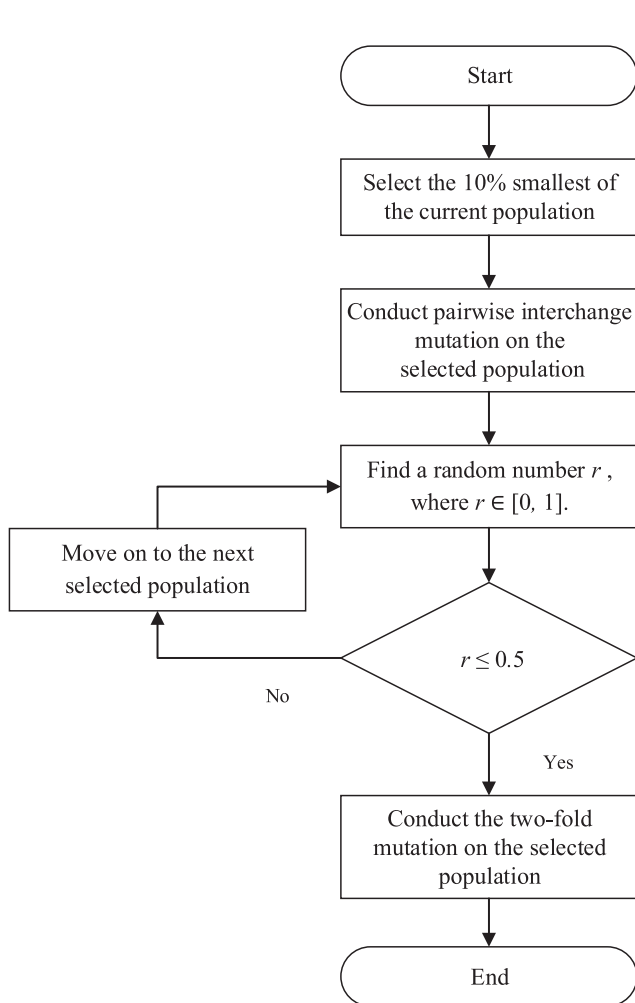


Fig. 9. Process of local search mutator.

Population	Percentage (%)
Previous Population	100

↓

Population	Percentage (%)
Top 10% chromosomes of the previous population	10
Offspring of previous population (Generated only by top 10% chromosomes)	80
Chromosomes generated from Local Search Mutator	10

Fig. 10. Percentage alteration of the replacement.

values will be selected to participate in reproduction of next generation by the elite rule. Thus, low fitness strings feature a chance to achieve a high number of copies selected. Two chromosomes are then randomly selected, and they will be used in crossover and mutation.

The selected chromosomes are recombined to generate offspring using order-preserving one-point crossover according to the crossover rate. As Werner (2011) mentioned, the order-preserving one-point crossover determines a cut point in the beginning and then selects a segment until the cut point from the first parent; the missing indices are later inserted after the cut point in the relative order of the second parent. All genes behind the cut point are swapped between parent chromosomes, thereby rendering a new child chromosome. The characteristic of order-preserving one-point crossover is the nonrepetitiveness of every operation sequence. In other words, every chromosome generation is unique.

Fig. 8 shows an example of the order-preserving one-point crossover. In the example, the selected cut point of parent chromosome 1 is

Table 3
Fuzzification table.

$\Delta f_{avg}(t)$ or $\Delta f_{avg}(t-1)$									
$x \leq -0.7$		$-0.7 \leq x \leq -0.5$	$-0.5 \leq x \leq -0.3$	$-0.3 \leq x \leq -0.1$	$-0.1 \leq x \leq 0.1$	$0.1 \leq x \leq 0.3$	$0.3 \leq x \leq 0.5$	$0.5 \leq x \leq 0.7$	$x \geq 0.7$
i, j	-4	-3	-2	-1	0	1	2	3	4
Noted: $x = \Delta f_{avg}(t-1)$ or $\Delta f_{avg}(t)$; i or j is the output of $\Delta f_{avg}(t-1)$ and $\Delta f_{avg}(t)$									

Table 4
Defuzzification table.

$Z(i, j)$		i								
		-4	-3	-2	-1	0	1	2	3	4
j	-4	-4	-3	-3	-2	-2	-1	-1	0	0
	-3	-3	-3	-2	-2	-1	-1	0	0	1
	-2	-3	-2	-2	-1	-1	0	0	1	1
	-1	-2	-2	-1	-1	0	0	1	1	2
	0	-2	-1	-1	0	0	1	1	2	2
	1	-1	-1	0	0	1	1	2	2	3
	2	-1	0	0	1	1	2	2	3	3
	4	0	1	1	2	2	3	3	4	4

6. Hence, the genes after cut point 6 are replaced by selected genes of parent chromosome 2. Eventually, children chromosome 1 is generated, and the crossover process will cease until sufficient children chromosomes are generated.

5.3. Pairwise interchange mutation and LS mutator

In this research, we utilize the pairwise interchange mutation, where two arbitrary inbound trailers are selected and interchanged in the chromosome, as mentioned by Werner (2011). LS is applied probabilistically in AGA. In other words, whether to apply a simple or LS mutator is decided dynamically in the process of the proposed AGA. In this research, the probability of using LS mutator is set at 0.5, as recommended by previous research. Most importantly, the main difference between the simple mutation and LS mutator is twofold (Ombuki & Ventresca, 2004). Exactly one gene swap is allowed in the normal mutation process. Thus, the positions of two randomly selected inbound trailers are swapped once. However, by applying the LS mutator in this algorithm, a systematic approach is used to consecutively swap the genes of selected chromosomes. First, we select the best 10% chromosomes of the initial population. Second, pairwise interchange mutation is performed on the selected chromosomes. Finally, a random number r is generated. If r is lower than 0.5, then two-fold mutation will be conducted. Fig. 9 presents the steps of the LS phase.

5.4. Replacement and termination criteria

To enhance the performance of each generation, the top 10% chromosomes of the previous population are selected for the next generation. The proposed AGA selects the top 10% chromosomes to

undergo crossover and mutation processes and forms 80% of new chromosomes for the next generation. Finally, the remaining 10% of chromosomes comprise those that have been through the LS mutator. The replacement of chromosomes follows the percentage alteration shown in Fig. 10.

The operation and replacement processes of this algorithm are repeated until certain termination criteria have been reached. AGA includes a number of termination criteria, and the maximum number of generations is the proposed termination criteria adopted in this research. The proposed algorithm will conclude and finish computing by completing the predetermined maximal iterations.

5.5. Heuristic FLC algorithm

To adaptively regulate GA operators using FLC, this research utilizes the FLC scheme proposed by Yun and Gen (2003). The main idea of the fuzzy logic controller in AGA involves the crossover and mutation FLC, which are implemented independently to adaptively regulate crossover and mutation rates during the process. By considering the changes in average fitness of two generations in the GA, this heuristic strategy can update the crossover and mutation rates. In other words, if the current chromosome consistently produces better offspring during the GA process, then this heuristic increases the P_c and P_m of the operator's occurrence correspondingly. As this research aims to minimize the makespan of the sorting process, we can set the change in average fitness value at generation g as $\Delta f_{avg}(g)$, which was used in a previous study (Yun & Gen, 2003). The equation is shown as follows:

$$\Delta f_{avg}(g) = (f_{par_size}(g) - f_{off_size}(g)) \times \lambda$$

$$= \left(\frac{\sum_{k=1}^{par_size} f_k(g)}{par_size} - \frac{\sum_{k=par_size+1}^{par_size+off_size} f_k(g)}{off_size} \right) \times \lambda \quad (31)$$

where par_size and off_size denote the parent and offspring sizes, respectively. $f_{par_size}(g)$ and $f_{off_size}(g)$ are the average fitness values of parents and offspring in generation g , respectively. To apply defuzzification in FLC, λ is set to be a scaling factor to normalize average fitness, and its value varies according to the problem. $\Delta f_{avg}(g)$ and $\Delta f_{avg}(g+1)$ are used to regulate P_c and P_m , respectively, as shown by the following (Yun & Gen, 2003):

Table 5
Detail contents of each approach.

Approach abbreviation	Detail of approach content
GA	Traditional GA with order preserving one-point crossover and pairwise interchange mutation
LSGA	Genetic algorithm combining a local search two-fold mutator is used in order to prevent GA from being trapped in local optimal solution
FLCGA	Genetic algorithm with the utilization of fuzzy logic control to adjust crossover rate and mutation rate
AGA	Both the local search and fuzzy logic control are added on the GA

Table 6
Comparison of processes and mechanisms between algorithms.

Process	Mechanism			
	GA	LSGA	FLCGA	AGA
Randomly generate initial population	✓	✓	✓	✓
Evaluation of fitness value	✓	✓	✓	✓
Encoding and selection of GA	✓	✓	✓	✓
Crossover and selection of GA	✓✓	✓	✓	✓
FLC	×	×	✓ (Automatically modify the crossover and mutation rate)	✓ (Automatically modify the crossover and mutation rate)
LS	×	✓ (Local Search through two-fold mutation)	×	✓ (Local Search through two-fold mutation)

Table 7
Parameter setting for each algorithm.

Algorithm	GA	LSGA	FGA	HGA
Population size	80	80	80	80
Crossover rate	0.8	0.8	0.8 (Initial)	0.8 (Initial)
Mutation rate	0.05	0.05	0.05 (Initial)	0.05 (Initial)

Table 8
Comparison between simple layout and complex layout.

Type of layout	Details
Simple layout	- One Mainline Conveyor (Fourteen conveyor segments) - Two Unload Docks - Four Load Docks - Four Shortcuts - Fourteen Routes in total
Complex layout	- Two Mainline Conveyors (Twenty-four conveyor segments) - Four Unload Docks - Eight Load Docks - Six Shortcuts - Eighty-one Routes in total

Procedure: Regulating Strategy of FLC

1	Input: $\Delta f_{avg}(g-1)$ and $\Delta f_{avg}(g)$.
2	If $\varepsilon \leq \Delta f_{avg}(g-1) \leq \gamma$ and $\varepsilon \leq \Delta f_{avg}(g) \leq \gamma$, then
3	increase P_m and P_m for next generation;
4	Else if $-\gamma \leq \Delta f_{avg}(g-1) \leq -\varepsilon$ and $-\gamma \leq \Delta f_{avg}(g) \leq -\varepsilon$, then
5	decrease P_m and P_m for next generation;
6	Else if $-\varepsilon \leq \Delta f_{avg}(g-1) \leq \varepsilon$ and $-\varepsilon \leq \Delta f_{avg}(g) \leq \varepsilon$, then
7	rapidly increase P_c and P_m for next generation;
8	End if;
9	Output: P_c and P_m .

This research uses the fuzzy decision table mentioned by Song, Wang, Wang, and Johns (1997); it is based on numerous experiments and domain expert options. Lin and Gen (2009) claimed that the input values are normalized to integer values from -4 to 4 and according to their corresponding minimum and maximum values. Table 3 displays the fuzzy decision table values.

For simplicity, the defuzzification table for determining the control action of crossover and mutation FLCs is provided, as shown in Table 4 (Song et al., 1997). After normalizing $\Delta f_{avg}(g-1)$ and $\Delta f_{avg}(g)$, FLC assigns $\Delta f_{avg}(g-1)$ and $\Delta f_{avg}(g)$ to i and j , respectively, corresponding to the control action from the defuzzification table.

Hence, the changes in $\Delta c(g)$ and $\Delta m(g)$ are respectively calculated as follows:

$$\Delta c(g) = Z(i, j) * C_c \quad (32)$$

$$\Delta m(g) = Z(i, j) * C_m \quad (33)$$

where $Z(i, j)$ refers to the adjusted value. $C_c (= 0.02)$ and $C_m (= 0.002)$ are given to regulate the increasing and decreasing ranges of crossover and mutation rates, respectively. Finally, P_c and P_m are renewed by Eqs. (34) and (35), respectively. The adjusted rates should remain between 0.5 and 1.0 for $P_c(g)$ and 0.0 to 0.1 for $P_m(g)$ (Yun & Gen, 2003).

$$P_c(g) = P_c(g-1) + \Delta c(g) \quad (34)$$

$$P_m(g) = P_m(g-1) + \Delta m(g) \quad (35)$$

6. Computational study

6.1. Design of experiments

This section will first elaborate on all factors considered in this study. Then, consequences of the design of experiment (DOE) will be revealed to show the significant factors.

• Algorithm factor

Table 5 provides the details of each algorithm. AGA is the proposed algorithm, which adds LS and FLC to the traditional GA. Table 6 clarifies the differences in processes and mechanisms between GA, LSGA, FLCGA, and AGA. All these GAs follow the same manner of generating initial population, evaluating fitness value, and encoding and selecting chromosomes. These GAs equally utilize crossover and mutation methods, including order-preserving one-point crossover and pairwise interchange mutation, respectively. The main difference among these GAs is the exploitation of LS and FLC.

Parameter setting is a crucial issue in the searching procedure of the algorithm that determines the correct direction and derives the global optimal. In order to determine the suitable control parameters of the algorithms, we first conducted the full factorial design of experiments to identify the optimal settings for the control parameters of the proposed AGA algorithm as shown in Table 7. Based on these optimal settings, four types of GAs are solved and compared. The crossover rate and mutation rate are also the same for all approaches; however, the crossover rate (0.8) and mutation rate (0.05) in FLCGA and AGA are only serve as initial value, their crossover rate and mutation rate will be fine-tuning according to the changes of average fitness in two consecutive generations. That is, in the parameters of the FLCGA and AGA, all the parameters except for crossover rate and mutation rate were fixed during the searching process. In addition, the termination criterion, maximum number of generations, is set as 100 since these algorithms are converged within 100 generations as shown from Figs. 14–16.

• Layout factor

Two types of sortation system layouts (see Table 8) are utilized in

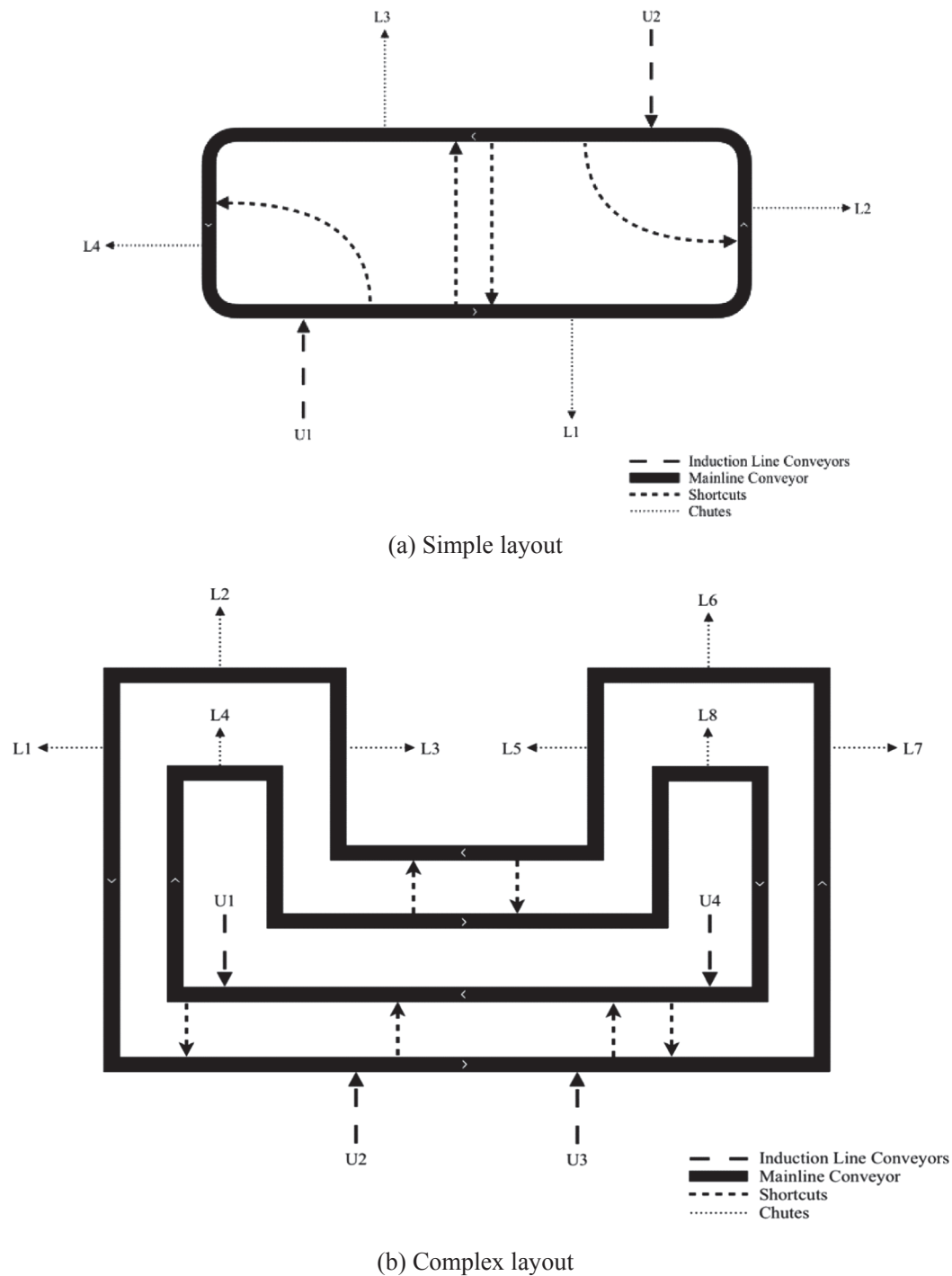


Fig. 11. Illustration of the simple layout and complex layout.

Table 9
Details of three different quantities of parcels in the simple layout.

Loading	Quantity of the parcels in each inbound trailer	Quantity of the inbound trailer
Small (15,000)	100	50
	100	50
	200	50
	300	50
Large (50,000)	100	50
	200	50
	300	50
	400	50

the computational study. The first layout is a simple layout, which is shown in Fig. 11(a). The simple layout contains one mainline conveyor, two unloading docks, four loading docks, and four shortcuts. The second layout is complex layout, which is shown in Fig. 11(b). The complex layout contains two mainline conveyors, four unloading docks, eight loading docks, and six shortcuts.

• Loading factor

Two levels of load distribution—even and uneven—are utilized in DOE. For even load distribution, parcels will be allocated evenly across loading docks. For instance, each loading dock in the simple layout will

Table 10
Details of three different quantities of parcels in the complex layout.

Loading	Quantity of the parcels in each inbound trailer	Quantity of the inbound trailer
Small (30,000)	100	50
	200	50
	300	50
Medium (50,000)	100	50
	200	50
	300	50
	400	50
Large (75,000)	100	50
	200	50
	300	50
	400	50
	500	50

Table 11
Factors and levels of the DOE.

Factors	Levels			
	GA	LSGA	FLCGA	AGA
Algorithms				
Layout	simple	complex		
Loading (Depend on Layout)	small	medium	large	
Batch Size	equal	unequal		
Arrival time	even	uneven		
Load Distribution	even	uneven		

receive approximately 25% of the total parcels. As for uneven load distribution, one of the loading docks will be allocated 50% of the total parcels. For example, loading dock 1 in the simple layout will receive at least half of the total parcels. Tables 9 and 10 show the details of three different loadings for simple layout and load distribution, respectively. Table 10 shows the details of three different loadings for the complex layout. All details shown in both tables are in the condition of unequal batch sizes.

- Batch size factor

Two levels of batch size—equal and unequal—are utilized in DOE. Tables 9 and 10 provide the details for unequal batch sizes.

- Arrival time factor

Two levels of arrival time—even and uneven—are utilized in DOE. For even arrival time, all inbound trailers will arrive at the first time slot. Hence, all inbound trailers are available at the beginning. As for the uneven arrival time, inbound trailers arrive uniformly in the first 10 time slots.

- Load distribution factor

Two levels of load distribution—even and uneven—are utilized in DOE. For even load distribution, every loading dock will accommodate approximately the same number of parcels. For instance, each loading dock in the simple layout will receive approximately 25% of the total parcels. As for uneven load distribution, one loading dock will be allocated with 50% of the total parcels. For example, loading dock 1 in the simple layout will receive at least half of the total parcels.

6.1.1. Results of DOE

In this section, DOE is conducted to test whether the factors can significantly affect the algorithm results. Table 11 shows all the factors and their DOE levels. Six factors, including algorithm, layout, loading, batch size, arrival time, and load distribution, are considered in analysis.

Experiments are performed on a computer workstation with an i5-7500 CPU, 16 GB RAM, and Microsoft Windows 10 operating system. The response factor is completion time of the sorting process. A general full factorial design is conducted via Minitab. Every experiment will be replicated five times, resulting in 960 experiments in total.

Fig. 12 shows the residual plots for completion time. The two figures on the left-hand side indicate that the data follow a normal distribution. The two figures on the right-hand side show that the data follow no apparent trend. Therefore, the data are suitable for DOE. Table 12 illustrates the results of the DOE. The system performance possesses a predicting capability as R-square values are above 85%. Thus, all

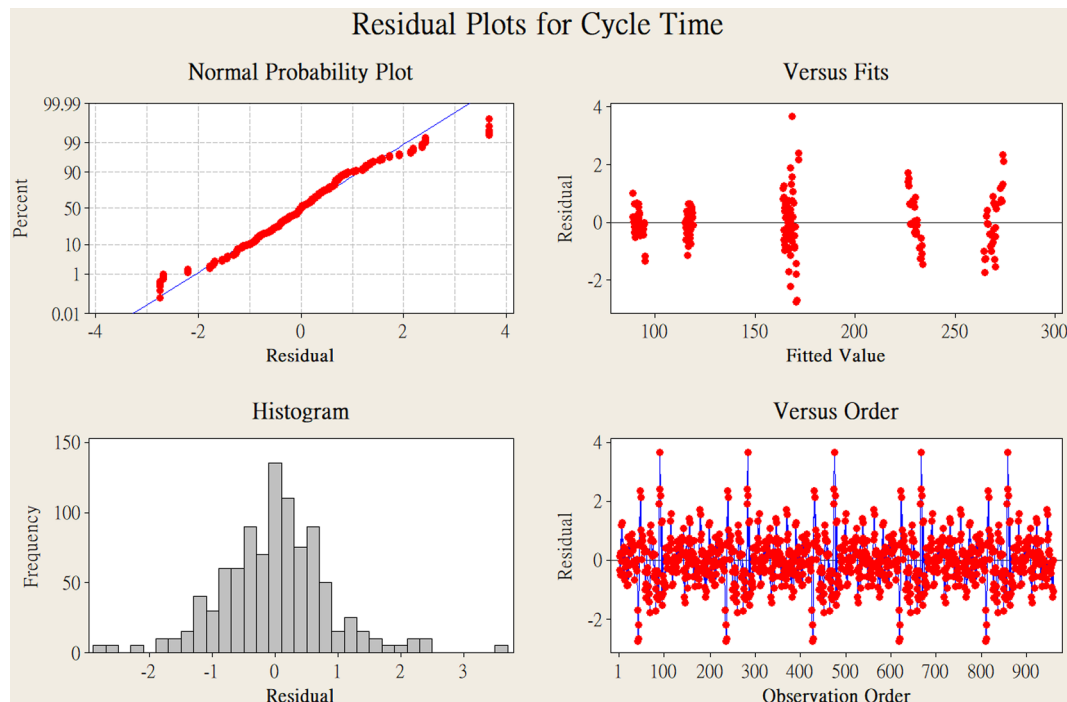


Fig. 12. Residual plots for completion time.

Table 12
ANOVA table of the DOE.

Source	DF	Seq SS	Adj SS	Adj MS	F	P
Layout	1	6090	6090	6090	7529.41	0.000*
Batch Size	1	4	4	4	5.09	0.024*
Arrival Time	1	21	21	21	25.4	0.000*
Load Distribution	1	1584	1584	1584	1958.47	0.000*
Loading	2	3,400,903	3,400,903	1,700,452	2,102,382	0.000*
Algorithms	3	1804	1804	601	743.5	0.000*
Layout*Batch Size	1	1	1	1	1.64	0.200
Layout*Arrival Time	1	16	16	16	20.39	0.000*
Layout*Load Distribution	1	213	213	213	22.97	0.000*
Layout>Loading	2	167,470	167,470	83,735	103527.2	0.000*
Layout*Algorithms	3	13	13	4	5.24	0.001*
Batch Size*Arrival Time	1	5	5	5	66.69	0.010*
Batch Size*Load Distribution	1	14	14	14	17.56	0.000*
Batch Size>Loading	2	82	82	41	50.77	0.000*
Batch Size*Algorithms	3	2	2	1	0.91	0.437
Arrival Time*Load Distribution	1	4	4	4	4.66	0.031*
Arrival Time>Loading	2	9	9	5	5.55	0.004*
Arrival Time*Algorithms	3	13	13	4	5.49	0.001*
Load Distribution>Loading	2	182	182	91	112.21	0.000*
Load Distribution*Algorithms	3	8	8	3	3.42	0.017*
Loading*Algorithms	6	60	60	10	12.36	0.000*
Error	918	742	742	1		
Total	959	3,579,241				

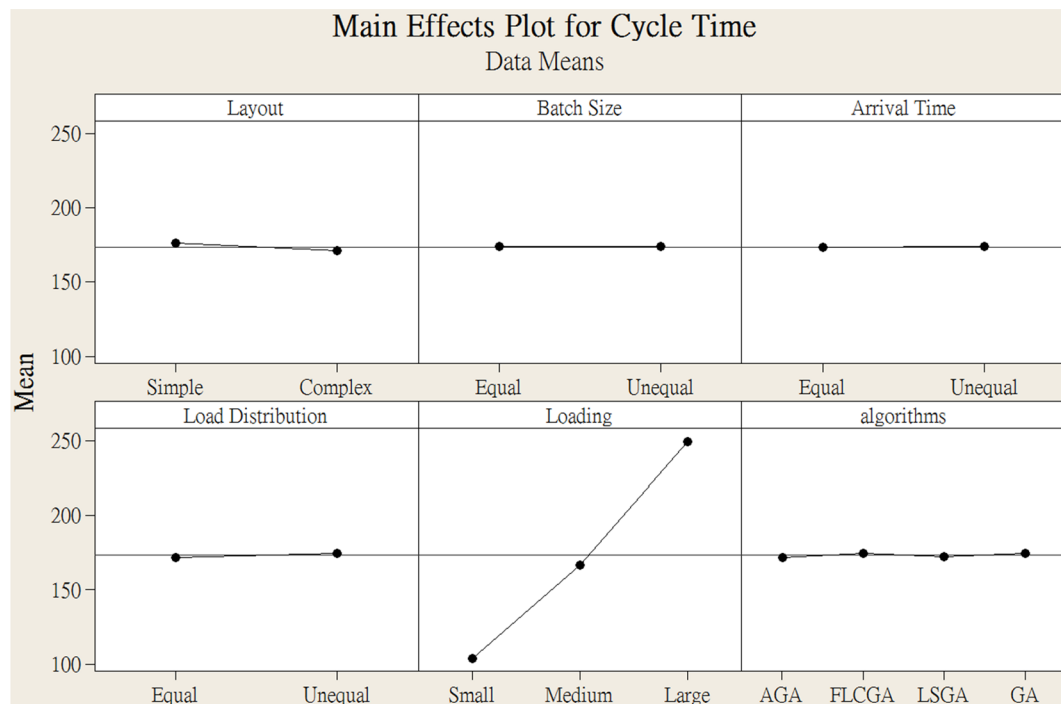


Fig. 13. Main effects plots for completion time.

Table 13
Experiment result for $CTR(a)$ of equal batch size and even arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	1.000	1.026	1.006	1.026	1.000	1.018	1.003	1.025
	Medium	1.000	1.011	1.003	1.011	1.000	1.015	1.007	1.019
	Large	1.000	1.008	1.003	1.008	1.000	1.016	0.116	1.017
Complex	Small	1.000	1.017	1.006	1.025	1.000	1.019	1.006	1.027
	Medium	1.000	1.022	1.003	1.023	1.000	1.021	1.003	1.025
	Large	1.000	1.009	1.002	1.016	1.000	1.010	1.002	1.015

Table 14
Experiment result for $CTR(a)$ of equal batch size and uneven arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	1.000	1.029	1.007	1.031	1.000	1.029	1.007	1.031
	Medium	1.000	1.017	1.006	1.19	1.000	1.016	1.007	1.020
	Large	1.000	1.01	1.004	1.013	1.000	1.020	1.005	1.020
Complex	Small	1.000	1.018	1.007	1.025	1.000	1.020	1.006	1.027
	Medium	1.000	1.022	1.004	1.023	1.000	1.020	1.006	1.026
	Large	1.000	1.009	1.002	1.013	1.000	1.010	1.004	1.013

Table 15
Experiment result for $CTR(a)$ of unequal batch size and even arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	1.000	1.019	1.007	1.029	1.000	1.019	1.009	1.029
	Medium	1.000	1.017	1.008	1.015	1.000	1.018	1.006	1.021
	Large	1.000	1.014	1.004	1.013	1.000	1.013	1.004	1.013
Complex	Small	1.000	1.017	1.013	1.028	1.000	1.019	1.006	1.022
	Medium	1.000	1.023	1.007	1.024	1.000	1.021	1.008	1.026
	Large	1.000	1.008	1.002	1.017	1.000	1.009	1.003	1.014

Table 16
Experiment result for $CTR(a)$ of unequal batch size and uneven arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	1.000	1.028	1.011	1.034	1.000	1.021	1.005	1.030
	Medium	1.000	1.023	0.01	1.024	1.000	1.026	1.015	1.026
	Large	1.000	1.019	1.004	1.019	1.000	1.024	1.005	1.026
Complex	Small	1.000	1.017	1.003	1.019	1.000	1.020	1.006	1.028
	Medium	1.000	1.022	1.004	1.022	1.000	1.020	1.008	1.025
	Large	1.000	1.009	1.002	1.013	1.000	1.009	1.003	1.017

Table 17
Overall solution result of $CTR(a)$ among four different algorithms.

	AGA	FLCGA	LSGA	GA
Overall	1.000	1.018	1.006	1.021

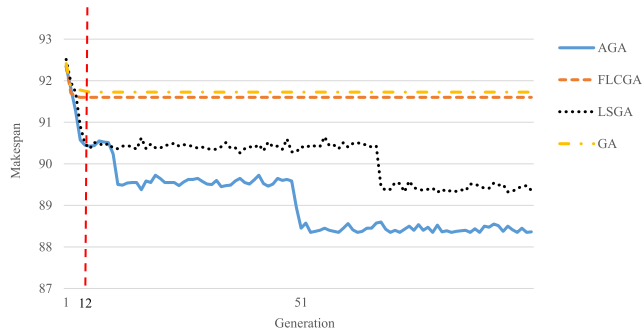


Fig. 14. Convergent condition of four algorithms in simple layout, small loading, unequal batch size, uneven arrival time, and even load distribution.

factors and most interactions between the two factors are significant.

Fig. 13 shows the main effect plot for the completion time of every factor. According to the figure, loading exerts the most significant impact on completion time. A larger load indicates a longer completion

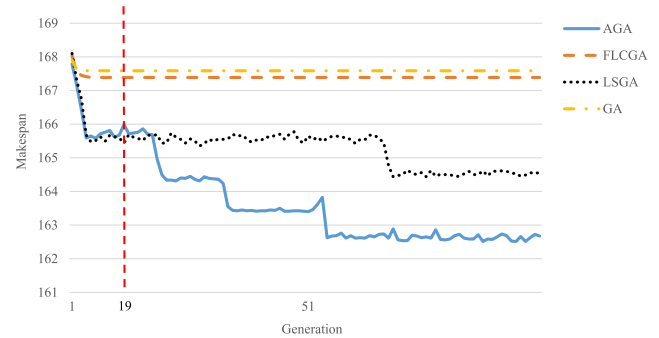


Fig. 15. Convergent condition of four algorithms in simple layout, medium loading, unequal batch size, uneven arrival time, and even load distribution.

time. As large load contains more inbound trailers than small ones, more parcels must be sorted. Load distribution also influences completion time. When load distribution is unequal, which indicates that plenty of parcels are headed to the same destination, congestion may occur in short routes. Hence, parcels must be allocated to longer routes. In consequence, the sorting process of every parcel will then be postponed. On the other hand, algorithms can also influence the completion time. Based on the figure, AGA and LSGA can relatively obtain shorter completion times than FLCGA and GA. In the next section, the four algorithms are compared to determine the algorithm that is most

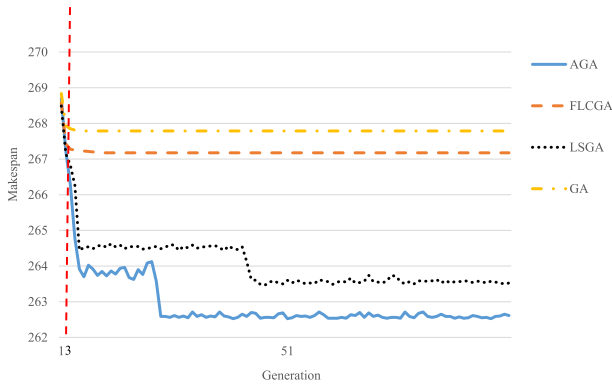


Fig. 16. Convergent condition of four algorithms in simple layout, large loading, unequal batch size, uneven arrival time, and even load distribution.

suitable to different scenarios.

6.2. Data analysis of different types of GAs

In the previous section, DOE is performed to recognize the influence of different factors on the output of experiments. In this section, we aim to distinguish improved algorithms among the four types of GAs by using key performance indices (KPIs), including quality ratio, convergent condition, run time, confidence interval, and box plot.

6.2.1. Experimental implementation

The quality of algorithms can be identified using different KPIs. In this research, for instance, a ratio of completion time acquired by a specific algorithm over the minimum completion time acquired by algorithms is used as one of the KPIs. This quality ratio CTR can be calculated as follows:

$$CTR(a) = \frac{CT(a, s)}{CT^m(s)} \quad \forall a \in A \quad (36)$$

where A denotes the set of algorithms; $CT(a, s)$ refers to the completion time by using algorithm a on scenario s ; $CT(a, s)$ corresponds to minimum completion time obtained by any algorithm on scenario s .

Other KPIs include the run time of algorithms $RT(s)$. Confidence interval $CI(s)$ is used to test the performance of different algorithms. Experiments on every scenario are replicated five times. Completion time, run time, and confidence interval of every algorithm are the average values obtained from the five replications.

6.2.2. Quality ratio

Tables 13–16 show the quality ratio of algorithms in different scenarios. The proposed AGA can obtain the best completion time as the quality ratio of AGA equals 1 in all scenarios. LSGA could also achieve better completion time than FLCGA and GA. FLCGA could still acquire solutions that are slightly better than those of GA. As a result, the

robustness of the proposed heuristic is evident because AGA outperforms other heuristics in every scenario. In general, AGA exhibits substantial improvement in small-load scenarios. Such an improvement decreases as loading expands.

Table 17 shows the average solution results of the four algorithms. The solution obtained by the AGA is 1.8% better than that obtained by FLCGA. The solution obtained by the AGA is 0.6% better than that obtained by LSGA. Finally, the solution obtained by the AGA is 2.1% better than that obtained by traditional GA.

6.2.3. Convergent condition comparison

The convergent conditions of certain scenarios are shown in the figures below. Fig. 14 shows the convergent conditions of four algorithms in simple layout, small loading, unequal batch size, uneven arrival time, and even load distribution. Fig. 15 shows the convergent conditions of four algorithms in simple layout, medium loading, unequal batch size, uneven arrival time, and even load distribution. Fig. 16 shows the convergent condition of four algorithms in simple layout, large loading, unequal batch size, uneven arrival time, and even load distribution.

In Fig. 14, AGA converges at the 50th generation. As shown in the red dotted line, the solutions are better than those after the 12th generation. In addition, LSGA performs better than FLCGA and GA. In Fig. 15, the AGA converges at the 55th generation. As shown in the red dotted line, the solutions are better than those after the 19th generation. Moreover, LSGA performs better than FLCGA and GA. In Fig. 16, the AGA converges at the 23rd generation. As shown in the red dotted line, the solutions are better than those after third generation. Furthermore, LSGA performs better than FLCGA and GA. Although FLCGA and GA can converge faster than AGA, the solutions obtained by the former two are substantially higher than those obtained by the latter.

In consequence, AGA converges to provide better solutions than the other algorithms. Furthermore, compared with other algorithms, AGA utilizes fewer generations to improve solutions. As a result, AGA performs better than the other algorithms when considering better solutions.

6.2.4. Run time

Tables 18–21 present the average run time comparison among GA, LSGA, FLCGA, and AGA of each scenario. In most scenarios, GA requires the least time to operate. For LSGA, FLCGA, and AGA, GA adds LS and FLC mechanisms that require additional time to operate the if-then decision and calculate the algorithm. The runtimes of AGA are higher than those of FLCGA and LSGA because it combines two mechanisms.

6.2.5. Confidence interval

Tables 22–25 show the experiment results of 95% confidence interval for every algorithm in each scenario. In most cases, the confidence intervals of AGA are smaller than those of other algorithms owing to its smaller standard variations. Moreover, FLCGA yields the smallest confidence interval in comparison with LSGA and GA. LSGA

Table 18
Experiment result for $RT(s)$ of equal batch size and even arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	375.85	373.49	272.23	371.43	508.53	414.97	525.54	408.69
	Medium	912.52	697.20	873.88	855.94	1,007.32	821.57	922.69	816.38
	Large	1,408.95	1,167.33	1,312.19	1,373.81	1,620.72	1,523.88	1,605.96	1,500.54
Complex	Small	8,756.12	8,236.85	8,614.94	7,833.70	9,102.46	8,588.96	8,831.76	8,217.56
	Medium	11,968.75	10,536.16	10,932.18	9,987.59	12,091.84	11,022.84	11,928.67	10,591.79
	Large	15,616.92	13,569.11	14,932.81	12,248.18	16,851.92	14,987.75	15,689.15	13,968.51

Unit: second.

Table 19
Experiment result for $RT(s)$ of equal batch size and uneven arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	399.82	373.85	396.47	383.07	535.99	423.03	505.12	141.66
	Medium	998.51	670.14	970.99	844.10	983.38	670.14	970.99	844.14
	Large	1,623.76	1,402.32	1,611.81	1,351.67	1,642.78	1,402.32	1,611.81	1,438.32
Complex	Small	8,801.28	8,298.22	8,704.58	7,963.91	9,072.75	8,558.36	8,793.27	8,205.38
	Medium	11,993.85	10,598.12	11,082.31	10,329.77	11,953.76	10,937.79	11,843.86	10,507.45
	Large	15,898.16	14,385.53	15,571.96	13,921.86	16,802.76	15,002.12	15,697.63	14,012.52

Unit: second.

Table 20
Experiment result for $RT(s)$ of unequal batch size and even arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	473.84	314.05	464.69	393.57	469.94	338.03	450.75	416.51
	Medium	874.58	675.03	831.12	817.40	864.52	817.78	854.53	802.40
	Large	1,389.16	1,167.33	1,340.87	1,072.16	1,368.72	1,159.16	1,340.69	1,121.04
Complex	Small	8,758.76	8,337.15	8,645.63	7,901.43	9,273.43	8,637.75	8,957.74	8,596.44
	Medium	11,872.43	10,499.32	10,965.21	10,188.96	12,018.95	10,995.88	11,882.34	10,585.02
	Large	15,769.47	14,277.44	15,333.70	13,699.91	16,801.37	14,983.06	15,656.75	13,958.33

Unit: second.

Table 21
Experiment result for $RT(s)$ of unequal batch size and uneven arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	462.88	381.94	409.27	365.94	462.88	381.94	409.37	365.94
	Medium	861.84	798.22	848.91	792.37	849.28	767.85	811.91	762.63
	Large	1,298.63	1,137.69	1,247.59	1,118.89	1,658.31	1,566.80	1,516.71	4,797.03
Complex	Small	8,835.47	8,518.20	8,704.82	8,315.24	9,195.75	8,535.76	8,957.74	8,596.44
	Medium	11,845.65	10,467.16	10,923.64	10,149.38	12,018.95	10,995.88	11,882.34	10,582.02
	Large	15,792.37	14,056.78	14,553.22	12,248.18	15,834.91	14,827.67	15,358.24	13,452.67

Unit: second.

Table 22
Experiment results for $CI(s)$ of equal batch size and even arrival time.

Layout	Load Distribution	Even				Uneven			
		AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	(89.28, 89.52)	(91.62, 91.85)	(89.84, 90.09)	(91.6, 91.9)	(92.09, 92.37)	(93.75, 94.07)	(92.37, 92.63)	(94.29, 94.73)
	Medium	(164.82, 165.17)	(166.58, 166.88)	(165.3, 165.3)	(166.69, 167.08)	(166.11, 166.51)	(168.5, 168.97)	(167.24, 167.81)	(169.2, 169.64)
	Large	(265.83, 266.23)	(267.77, 268.5)	(266.33, 267.29)	(267.77, 268.5)	(268.38, 268.67)	(273.54, 274.3)	(270.89, 271.23)	(273.76, 274.45)
Complex	Small	(114.58, 114.76)	(116.51, 116.84)	(115.27, 115.48)	(117.41, 117.75)	(116.05, 116.32)	(118.19, 118.52)	(116.69, 116.98)	(119.19, 119.5)
	Medium	(163.47, 163.76)	(167.11, 167.46)	(163.92, 164.29)	(167.14, 167.65)	(165.58, 165.92)	(169, 169.35)	(165.98, 166.38)	(169.61, 170.08)
	Large	(227.42, 227.75)	(229.5, 229.89)	(227.84, 228.18)	(231.09, 231.49)	(229.38, 229.77)	(231.72, 232.19)	(229.78, 230.23)	(232.8, 233.31)

Unit: second.

and GA approximately obtain the same confidence interval because of equal standard deviations. However, confidence interval increases when the loading of a scenario expands. According to the results, AGA outperforms other algorithms.

The distribution of solutions could be clarified via box plot. The box

plots of certain scenarios are shown in the following figures. Figs. 17–19 show that the AGA in these scenarios can find a smaller average completion time than the other algorithms. In addition, solutions obtained via AGA present a smaller deviation than others. Overall, given the excellent performance of the proposed AGA in every aspect, it

Table 23
Experiment results for $CI(s)$ of equal batch size and uneven arrival time.

Layout	Load Distribution	Even				Uneven					
		Algorithms Loading		AGA	FLCGA	LSGA	GA	FLCGA	LSGA	GA	
Simple	Small			(89.49, 89.83)	(92.04, 92.56)	(89.98, 90.62)	(92.34, 92.62)	(91.69, 92.03)	(94.24, 94.76)	(92.2, 92.84)	(94.54, 94.82)
	Medium			(164.7, 165.01)	(167.6, 167.85)	(165.55, 166.12)	(167.88, 167.93)	(164.95, 165.16)	(167.6, 167.85)	(165.85, 166.42)	(168.28, 168.33)
	Large			(265.47, 266.18)	(267.81, 269.02)	(166.46, 267.18)	(268.74, 269.72)	(270.55, 271.18)	(275.88, 276.55)	(271.9, 272.43)	(275.89, 276.92)
Complex	Small			(114.71, 114.96)	(116.73, 117.02)	(115.51, 115.78)	(117.45, 117.9)	(115.86, 116.18)	(118.13, 118.5)	(116.62, 116.88)	(118.94, 119.32)
	Medium			(163.62, 163.97)	(167.14, 167.58)	(164.16, 164.65)	(167.28, 167.79)	(165.33, 165.82)	(168.77, 169.16)	(166.27, 166.8)	(169.57, 170.13)
	Large			(227.61, 228.02)	(229.64, 230.2)	(228.08, 228.6)	(230.52, 231.23)	(229.17, 229.62)	(231.24, 231.91)	(230.02, 230.66)	(231.96, 232.59)

Unit: second.

Table 24
Experiment results for $CI(s)$ of unequal batch size and even arrival time.

Layout	Load Distribution	Even				Uneven					
		Algorithms Loading		AGA	FLCGA	LSGA	GA	FLCGA	LSGA	GA	
Simple	Small			(89.76, 90.06)	(91.38, 91.84)	(89.99, 90.97)	(92.35, 92.75)	(91.73, 92.09)	(93.38, 93.84)	(92.33, 93.09)	(94.35, 94.75)
	Medium			(164.24, 164.91)	(167.01, 167.78)	(165.46, 166.2)	(166.76, 167.47)	(165.78, 166)	(168.74, 169.03)	(166.78, 166.99)	(169.33, 169.58)
	Large			(263.2, 263.52)	(266.68, 267.31)	(263.88, 265.14)	(266.1, 267.36)	(269.8, 270.11)	(273.27, 273.9)	(270.87, 271.18)	(273.02, 273.66)
Complex	Small			(114.42, 114.64)	(116.3, 116.73)	(115.85, 116.09)	(117.61, 117.95)	(115.85, 116.11)	(117.99, 118.45)	(116.52, 116.87)	(118.24, 118.74)
	Medium			(163.35, 163.77)	(167.13, 167.51)	(164.52, 165.03)	(167.31, 167.73)	(165.23, 165.69)	(168.61, 169.16)	(166.6, 167.09)	(169.47, 170.04)
	Large			(227.47, 228.02)	(229.28, 230.03)	(227.84, 228.54)	(231.17, 231.92)	(228.83, 229.29)	(230.77, 231.49)	(229.43, 230.07)	(231.81, 232.56)

Unit: second.

Table 25
Experiment results for $CI(s)$ of unequal batch size and uneven arrival time.

Layout	Even				Uneven			
	Load Distribution				Algorithms			
	AGA	FLCGA	LSGA	GA	AGA	FLCGA	LSGA	GA
Simple	Small	(89.02, 89.23)	(91.47, 91.75)	(90.03, 90.26)	(91.34, 91.68)	(93.22, 93.65)	(91.74, 92.24)	(94.05, 94.53)
	Medium	(163.48, 163.88)	(167.17, 167.62)	(165.03, 165.54)	(169.23, 169.61)	(173.68, 174.13)	(171.75, 172.22)	(173.49, 174.03)
Complex	Large	(262.75, 263.22)	(266.93, 267.46)	(263.82, 264.33)	(268.18, 268.58)	(274.14, 275.27)	(269.07, 270.29)	(274.73, 275.6)
	Small	(114.53, 114.77)	(116.41, 116.9)	(114.81, 115.16)	(115.89, 116.12)	(118.19, 118.46)	(116.48, 116.96)	(119, 119.39)
	Medium	(163.65, 164.1)	(167.16, 167.69)	(164.39, 164.81)	(165.38, 165.81)	(168.71, 169.11)	(166.6, 167.11)	(169.5, 170.05)
	Large	(227.65, 228.12)	(229.59, 230.16)	(228.15, 228.76)	(228.95, 229.4)	(230.95, 231.64)	(229.57, 230.25)	(232.86, 233.45)

Unit: second.

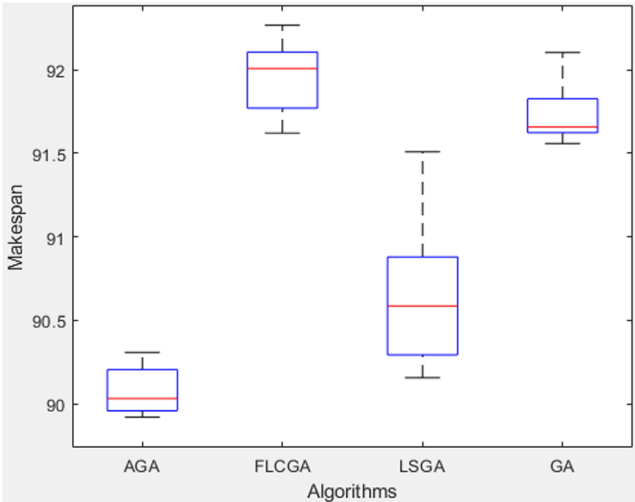


Fig. 17. Box plot of four algorithms in simple layout, small loading, unequal batch size, even arrival time, and even load distribution.

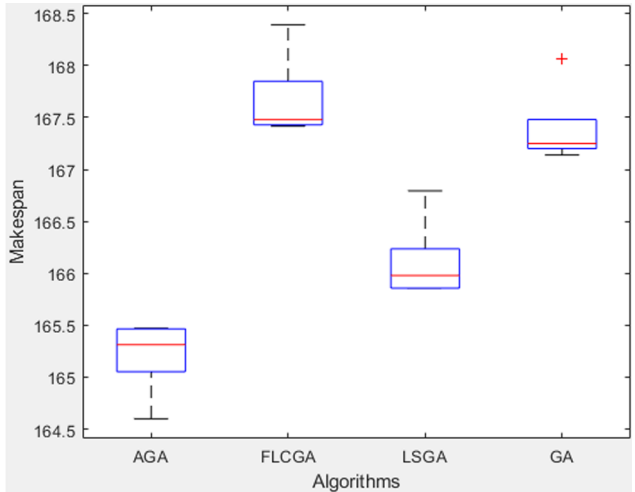


Fig. 18. Box plot of four algorithms in simple layout, medium loading, unequal batch size, even arrival time, and even load distribution.

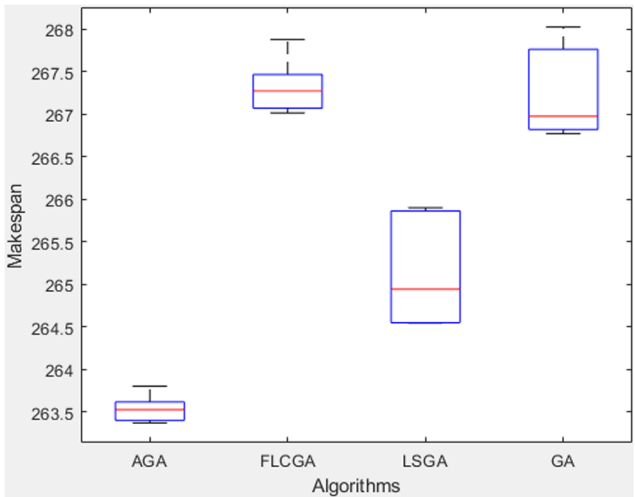


Fig. 19. Box plot of four algorithms in simple layout, large loading, unequal batch size, even arrival time, and even load distribution.

Table 26

Average makespan between shortcut-free layout and shortcuts-based layout under different scenarios of the simple layout.

Layout Scenario	Load Distribution	Even				Uneven			
		Even		Uneven		Even		Uneven	
	Arrival Time Loading	Even	Uneven	Even	Uneven	Even	Uneven	Even	Uneven
Shortcut-free Layout	Small	110.33	109.93	110.34	111.95	111.18	110.97	112.40	112.85
	Medium	187.46	186.90	184.89	184.89	187.25	187.31	186.59	186.59
	Large	283.38	282.54	281.03	283.04	283.10	272.41	280.50	281.74
Shortcuts-based Layout	Small	89.40	89.66	89.91	93.63	92.23	91.86	91.91	91.51
	Medium	165.00	164.86	164.58	165.18	166.31	165.06	165.89	169.42
	Large	266.03	265.83	263.36	262.99	269.53	270.87	269.96	268.38

Unit: second.

Table 27

Improvement rates of shortcuts-based layout.

Load Distribution		Even				Uneven			
Batch Size		Even		Uneven		Even		Uneven	
Arrival Time		Even	Uneven	Even	Uneven	Even	Uneven	Even	Uneven
Loading	Small	23.41%	22.60%	22.72%	19.57%	20.55%	20.80%	22.29%	23.31%
	Medium	13.62%	13.37%	12.34%	11.93%	12.59%	13.49%	12.48%	10.13%
	Large	6.52%	6.29%	6.71%	7.62%	5.04%	0.57%	3.91%	4.98%

is considered an effective and efficient approach for resolving PHSPwS.

6.3. Benefit of parcel hub scheduling problem with shortcuts

This section shows the benefit of parcel hub scheduling problem with shortcuts (PHSPwS) compared to previous PHSP problem (shortcut-free layout). The utilization of shortcuts in the closed-loop sortation system (shortcuts-based layout) not only improve the speed and efficiency of parcel sorting but also increase throughput by expanding the system capacity. However, there were not enough previous research to confirm this novel idea. This research provides an assessment to demonstrate the enormous benefits of the utilization of shortcuts by running experiments with the AGA algorithm under different scenarios of the simple layout. Table 26 shows average makespan of five replications between shortcut-free layout and shortcuts-based layout under different scenarios of the simple layout. For all scenarios, the shortcuts-based layout outperforms the shortcut-free layout. Table 27 shows that shortcuts-based layout can greatly improve the sorting efficiency and decrease the makespan by more than 10%.

7. Conclusion

This research addressed the PHSPwS problem featuring an unequal batch size, various arrival times of inbound trailers, and shortcuts on the closed-loop sortation system. PHSP is defined as the processing of numerous inbound trailers at a low number of unloading docks to find an unloading schedule to minimize the makespan of sorting process. This research has introduced the background of PHSPwS, variety of GA, and related literature. In addition, a mixed integer mathematical model is formulated to present the PHSPwS problem and its constraints. Moreover, an AGA utilizing FLC and LS is designed to solve the PHSPwS problem. Finally, the performance of the proposed algorithm was compared with that of other algorithms.

Several mechanisms have been adopted in AGA. After adding LS to the traditional GA, the algorithm can avoid entrapment in the local optimal solution. On the other hand, adding FLC to traditional GA could help the algorithm adjust the probability of crossover (P_c) and mutation (P_m) rates. Such an effect can be achieved by considering the change in

average fitness value of parents and offspring in two consecutive generations and can enhance the searching capability of the proposed algorithm.

The performance of each algorithm could be compared in detail by performing computational analysis among different types of GAs. DOE is conducted to identify whether all six factors can significantly affect the algorithm results. A total of 192 scenarios and 5 replications are conducted, thereby resulting in 960 experiments. KPIs, including quality ratio, convergent condition, run time, confidence interval, and box plot, are used to compare algorithms. Results show that LS and FLC can aid the GA to find a good solution and that AGA with LS and FLC can perform the solutions accordingly and with stability.

The contribution of this research is the resolution of PHSPwS using efficient and accurate methodologies. As the closed-loop sortation system is equipped with shortcuts, alternative routing is one of the issues in this research. Given the unequal batch size and various arrival times of inbound trailers, this PHSPwS problem is more complicated than those in previous research. Several mechanisms are combined into an algorithm to balance the exploration and exploitation of the search space to improve the evolution process.

This topic can be extended in following directions:

- Consider real issues, such as replacement time of trailers and various processing rates of trailers and unloading docks, in the parcel delivery industry;
- Improve the AGA proposed in this study or combine it with other algorithms to enhance the quality and robustness of results;
- Consider multi-objective problems rather than only optimize the makespan of the sorting process;
- Develop other algorithms apart from GA and compare the performance of each algorithm with that of others.

References

- Boysen, N., Briskorn, D., Fedtke, S., & Schmickerath, M. (2018). Automated sortation conveyors: A survey from an operational research perspective. *European Journal of Operational Research*, 276(3), 796–815.
- Ghiani, G., Laporte, G., & Musmanno, R. (2004). *Introduction to logistics systems planning*

- and control. John Wiley & Sons.
- Lai, J.-P. (2006). *Surrogate search: A simulation optimization methodology for large-scale systems*. University of Pittsburgh.
- Lin, L., & Gen, M. (2009). Auto-tuning strategy for evolutionary algorithms: Balancing between exploration and exploitation. *Soft Computing*, 13(2), 157–168.
- Masel, D. (1998). Adapting the longest processing time heuristic for output station assignment in a transfer facility. In Proceedings of the 1998 Industrial Engineering Research Conference.
- Masel, D., & Goldsmith, D. (1997). Using a simulation model to evaluate the configuration of a sortation facility. In Winter simulation conference.
- McWilliams, D. L. (2009a). A dynamic load-balancing scheme for the parcel hub-scheduling problem. *Computers & Industrial Engineering*, 57(3), 958–962.
- McWilliams, D. L. (2009b). Genetic-based scheduling to solve the parcel hub scheduling problem. *Computers & Industrial Engineering*, 56(4), 1607–1616.
- McWilliams, D. L. (2010a). A genetic based scheduling algorithm for the PHSP with unequal batch size inbound trailers. *Journal of Industrial and Systems Engineering*, 4(3), 167–182.
- McWilliams, D. L. (2010b). Iterative improvement to solve the parcel hub scheduling problem. *Computers & Industrial Engineering*, 59(1), 136–144.
- McWilliams, D. L., & McBride, M. E. (2012). A beam search heuristics to solve the parcel hub scheduling problem. *Computers & Industrial Engineering*, 62(4), 1080–1092.
- McWilliams, D. L., Stanfield, P. M., & Geiger, C. D. (2005). The parcel hub scheduling problem: A simulation-based solution approach. *Computers & Industrial Engineering*, 49(3), 393–412.
- McWilliams, D. L., Stanfield, P. M., & Geiger, C. D. (2008). Minimizing the completion time of the transfer operations in a central parcel consolidation terminal with unequal-batch-size inbound trailers. *Computers & Industrial Engineering*, 54(4), 709–720.
- Ombuki, B. M., & Ventresca, M. (2004). Local search genetic algorithms for the job shop scheduling problem. *Applied Intelligence*, 21(1), 99–109.
- Rohrer, M. (1995). *Simulation and cross docking. Proceedings of the 27th conference on Winter simulation*.
- Song, Y., Wang, G., Wang, P., & Johns, A. (1997). Environmental/economic dispatch using fuzzy logic controlled genetic algorithms. *IEE Proceedings-Generation, Transmission and Distribution*, 144(4), 377–382.
- Werner, F. (2011). Genetic algorithms for shop scheduling problems: a survey. Preprint, 11, p. 31.
- Yun, Y., & Gen, M. (2003). Performance analysis of adaptive genetic algorithms with fuzzy logic and heuristics. *Fuzzy Optimization and Decision Making*, 2(2), 161–175.